
第4章 分治算法

(Divide and Conquer)

主讲教师：刘春风
智能与计算学部
cfliu@tju.edu.cn

本章主要内容

□ 分治算法的基本思想

□ 分治算法的应用

- 1) 找伪币问题 2) 计算 a^n 3) 求Fibonacci数
- 4) 金块问题 5) 大整数乘法 6) 矩阵乘法
- 7) VLSI 布线 8) 残缺棋盘覆盖 9) 归并排序
- 10) 快速排序 11) 选择问题 12) 最接近点对问题
- 13) 循环赛日程

□ 复杂性的下限

- 最大最小问题的下限
 - 排序算法的下限
-

分治算法的思想

- 分治法设计算法的思想是：
 - 将问题分成(*devide*)多个子问题；
 - 递归地(*conquer*)解决每个子问题；
 - 将子问题的解合并(*combine*)成原问题的解；
 - 分治法常常得到递归算法.
 - Merge-Sort是用分治法设计算法的范例.
 - 算法复杂性分析
 - Master method
 - Substitution method
-

Review: Master method

- **递归方程:** $T(n)=aT(n/b)+f(n)$, 式中 $a \geq 1$, $b > 1$ 为整数, $f(n) > 0$ 。
- **情形1:** 若 $f(n)=O(n^{\log a - \epsilon})$, $\epsilon > 0$, 为某一常数, $f(n)$ 的增长渐近地慢于 $n^{\log a}$ (慢 n^ϵ 倍).

Solution: $T(n)=\Theta(n^{\log a})$.

- **情形2:** 若 $f(n)=\Theta(n^{\log a} \lg^k n)$, $k \geq 0$ 为某一常数, $f(n)$ 和 $n^{\log a}$ 几乎有相同的渐近增长率.

Solution: $T(n)=\Theta(n^{\log a} \lg^{k+1} n)$.

- **情形3:** 若 $f(n)=\Omega(n^{\log a + \epsilon})$, $\epsilon > 0$ 为一常数, $f(n)$ 多项式地快于 $n^{\log a}$ (by an n^ϵ factor), 且 $f(n)$ 满足以下规则性条件:

$af(n/b) \leq cf(n)$, $0 < c < 1$ 为常数.

Solution: $T(n) = \Theta(f(n))$.

适用条件

□ 分治法的适用条件

■ 分治法所能解决的问题一般具有以下几个特征：

- 问题的**规模缩小**到一定的程度就可以**容易地解决**；
- 问题可以分解为若干个规模较小的**相同问题**；
- 利用该问题分解出的子问题的解可以**合并**为该问题的解；
- 该问题所分解出的各个子问题是**相互独立**的，即子问题之间不包含公共的子问题。

■ 最后一条特征涉及到分治法的效率，如果各子问题是不独立的，则分治法要做许多不必要的工作，重复地解公共的子问题，此时虽然也可用分治法，但一般用**动态规划**较好。

步骤

■ 分治法的基本步骤

■ **divide-and-conquer(P)**

```
{  
  if ( | P | <= n0) then S(P); //解决小规模的问题  
  divide P into smaller subinstances P1,P2,...,Pk; //分解问题  
  for (i=1,i<=k,i++)  
    yi=divide-and-conquer(Pi); //递归的解各子问题  
  return merge(y1,...,yk); //将各子问题的解合并为原问题的解  
}
```

- 人们从大量实践中发现，在用分治法设计算法时，最好使子问题的规模大致相同。即将一个问题分成大小相等的 k 个子问题的处理方法是行之有效的。这种使子问题规模大致相等的做法是出自一种平衡(balancing)子问题的思想，它几乎总是比子问题规模不等的做法要好。
-

应用1：找伪币问题

- 现有16个硬币, 其中有一个是伪币, 并且伪币重量比真币轻。试用一台天平找出这枚伪币。
 - 两两比较, 最坏情形需8次比较找出伪币。
 - 分治法仅需4次比较找出伪币。
 - 是否还有更少的比较次数?
-

应用2：计算 a^n

- 问题：计算 a^n , 其中 $n \in \mathbf{N}$.
- Naive algorithm: $T(n) = \Theta(n)$.

应用2：计算 a^n

□ 分治算法：

□ $a^n = a^{n/2} \cdot a^{n/2}$ if n is even;
 $= a^{(n-1)/2} \cdot a^{(n-1)/2} \cdot a$ if n is odd.

□ 算法复杂度分析：

$$T(n) = T(n/2) + \Theta(1)$$

□ Master方法求解

Case 2: $\log_b a = 0, k = 0$

□ 算法复杂度：

$$T(n) = \Theta(\lg n)$$

应用3：求Fibonacci数

□ 问题：求Fibonacci数 F_n

■ 数列：1, 1, 2, 3, 5, 8, 13, 21, ...

■ 满足 $F_n = F_{n-1} + F_{n-2}$

■ 增加 $F_0=0$ ，得到数列：

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

□ 通常算法：

从 $F_0, F_1, \dots$, 根据定义陆续相加

□ 时间复杂度： $T(n) = \Theta(n)$

应用3：求Fibonacci数

□ 分治算法

设 $\{F^n\}$ 为Fibonacci数构成的数列，那么

$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

算法：令矩阵 $M = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$ ，用分治法计算 M^n

□ 时间复杂度： $T(n) = \Theta(\lg n)$

应用4：金块问题

- 有若干金块，试用一台天平找出其中最轻和最重的金块.
 - n 个数中找出最大和最小的数.

 - 直接求解方法1:
 - 先找出最大值, 然后在剩下的 $n-1$ 个数中再找出最小值.
 - 需 $2n-3$ 次比较.
-

应用4：金块问题

□ 直接求解方法2:

□ $\min \leftarrow a[0]; \max \leftarrow a[0];$

For $i \leftarrow 1$ to $n-1$ do

 if $a[i] < \min$

$\min \leftarrow a[i]$

 else if $a[i] > \max$

$\max \leftarrow a[i]$

□ 当输入数组未排序时,算法最多要做 $2(n-1)$ 次比较;
当输入为逆序时,算法要做 $n-1$ 次比较.

应用4：金块问题

□ 分治法求解：

□ $\text{max-min}(A[0, n-1], \text{max}, \text{min})$

if $n < 1$ $\text{max} \leftarrow \text{min} \leftarrow a[0], \text{return};$

else $m \leftarrow n/2$

$\text{max-min}(A[0, m-1], \text{max1}, \text{min1}),$

$\text{max-min}(A[m, n-1], \text{max2}, \text{min2}),$

$\text{max} \leftarrow \text{max}(\text{max1}, \text{max2}),$

$\text{min} \leftarrow \text{min}(\text{min1}, \text{min2}),$

$\text{return}.$

应用4：金块问题

□ 分治算法复杂度分析：

- 设 $c(n)$ 为使用分治法所需要的比较次数,假设 $n=2^k$ 则有:

$$c(n)=2c(n/2)+2, n \geq 2$$

$$c(n)=1, n = 1$$

□ 算法分析结果：

- 使用Master方法，解得 $c(n)=\Theta(n)$
- 通过迭代展开方法可解得： $c(n)= (3n/2)-2$

□ 分治算法与直接方法的比较

- 分治法比直接求解法少用了25%次比较

□ 可通过递归和非递归程序两种方式实现该问题

金块问题：求最小值和最大值的非递归程序

```
template<class T>
bool MinMax(T w[], int n, T& Min, T& Max)
// 寻找w[0:n-1]中的最小和最大值
// 如果少于一个元素，则返回 false
// 特殊情形：n <= 1
if (n < 1) return false;
if (n == 1) {Min = Max = 0;
              return true;}

//对Min 和Max进行初始化
int s; // 循环起点
```

```
if (n % 2) { // n 为奇数
    Min = Max = 0;
    s = 1;}
else { // n为偶数，比较第一对
    if (w[0] > w[1]) {
        Min = 1;
        Max = 0;}
    else {Min = 0;
          Max = 1;}
    s = 2;}
```


金块问题：求最小值和最大值的非递归程序

```
// 比较余下的数对
for (int i = s; i < n; i += 2) {
    // 寻找w[i] 和w[i+1]中的较大者
    // 然后将较大者与 w[Max]进行比较
    // 将较小者与 w[Min]进行比较
    if (w[i] > w[i+1]) {
        if (w[i] > w[Max]) Max = i;
        if (w[i+1] < w[Min]) Min = i + 1;
    }
    else {
        if (w[i+1] > w[Max]) Max = i + 1;
        if (w[i] < w[Min]) Min = i;
    }
}

return true;
}
```

算法分析

- 当 n 为奇数时, $n=2m+1$,比较 m 对相邻元素,

比较次数为 $3*m=3*(n-1)/2$

$$=3n/2-3/2=[3n/2]-1/2-3/2$$

$$=[3n/2]-2$$

[]表示向上取整.

- 当 n 为偶数时, $n=2m$,比较 $m-1$ 对相邻元素

比较次数为 $1+3*(m-1)=1+3*(n-2)/2$

$$=1+3n/2-3$$

$$=[3n/2]-2$$

- 该算法所用比较次数最少
-

应用5:大整数乘法

- 设计一个有效的算法，可以进行两个**n**位大整数的乘法运算

- 直接方法: $O(n^2)$ ✖效率太低

- 分治法:

- $X = 2^{n/2}A + B$

- $Y = 2^{n/2}C + D$

- $XY = 2^n AC + 2^{n/2} (AD + BC) + BD$

$$X = \overset{n/2\text{位}}{\boxed{A}} \overset{n/2\text{位}}{\boxed{B}} \quad Y = \overset{n/2\text{位}}{\boxed{C}} \overset{n/2\text{位}}{\boxed{D}}$$

- 复杂度分析: $T(n) = \begin{cases} O(1) & n = 1 \\ 4T(n/2) + O(n) & n > 1 \end{cases}$

- $T(n) = O(n^2)$ ✖没有改进☹

应用5:大整数乘法-算法改进

- 为了降低时间复杂度，一个可行的方法是减少乘法的次数。为此，把XY写成另外的形式：
 - $XY = 2^n AC + 2^{n/2} ((A-B)(D-C) + AC + BD) + BD$ 或
 - $XY = 2^n AC + 2^{n/2} ((A+B)(C+D) - AC - BD) + BD$
 - 复杂性：
 - 这两个算式看起来更复杂一些，但它们仅需要3次 $n/2$ 位乘法[AC、BD和 $(A \pm B)(C \pm D)$]，于是
 - $$T(n) = \begin{cases} O(1) & n = 1 \\ 3T(n/2) + O(n) & n > 1 \end{cases}$$
 - $T(n) = O(n^{\log 3}) = O(n^{1.59})$ ✓较大的改进😊
 - 细节问题：两个XY的复杂度都是 $O(n^{\log 3})$ ，但考虑到 $A+C, B+D$ 可能得到 $m+1$ 位的结果，使问题的规模变大，故不选择第2种方案。
-

更快的方法

- 直接方法: $O(n^2)$ ——效率太低
 - 分治法: $O(n^{1.59})$ ——较大的改进
 - 更快的方法?
 - 如果将大整数分成更多段, 用更复杂的方式把它们组合起来, 将有可能得到更优的算法。
 - 是否能找到线性时间的算法? 目前为止还没有结果。
-

应用6：矩阵乘法

- 两个 $n \times n$ 阶的矩阵A与B的乘积是另一个 $n \times n$ 阶矩阵C, C的元素为 $C(i, j) = \sum_k A(i, k)B(k, j)$

```
template <class T>
void MatrixMulti( T** A, T** B, T** C, int n)
{
    for (int i=0; i<n; i++)
        for (int j=0; j<n; j++){
            T sum=0;
            for (int k=0; k<n; k++)
                sum+=A[i][k]*B[k][j]
            c[i][j]=sum;
        }
}
```

- 每一个 $C(i, j)$ 都用此公式计算, 则计算C 所需要的运算次数为 n^3 次乘法和 $n^2(n-1)$ 次加法.
-

应用6：矩阵乘法-分治法

Divide-and-conquer algorithm

IDEA:

$n \times n$ matrix = 2×2 matrix of $(n/2) \times (n/2)$ submatrices:

$$\begin{bmatrix} r & s \\ t & u \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

$$C = A \cdot B$$

$$\left. \begin{array}{l} r = ae + bg \\ s = af + bh \\ t = ce + dh \\ u = cf + dg \end{array} \right\} \begin{array}{l} 8 \text{ mults of } (n/2) \times (n/2) \text{ submatrices} \\ 4 \text{ adds of } (n/2) \times (n/2) \text{ submatrices} \end{array}$$

应用6：矩阵乘法-分治法

Analysis of D&C algorithm

$$T(n) = 8T(n/2) + \Theta(n^2)$$

submatrices *submatrix size* *work adding submatrices*

$$n^{\log_b a} = n^{\log_2 8} = n^3 \Rightarrow \text{CASE 1} \Rightarrow T(n) = \Theta(n^3).$$

No better than the ordinary algorithm.

应用6: Strassen's矩阵乘法

Strassen's idea

- Multiply 2×2 matrices with only 7 recursive mults.

$$P_1 = a \cdot (f - h)$$

$$P_2 = (a + b) \cdot h$$

$$P_3 = (c + d) \cdot e$$

$$P_4 = d \cdot (g - e)$$

$$P_5 = (a + d) \cdot (e + h)$$

$$P_6 = (b - d) \cdot (g + h)$$

$$P_7 = (a - c) \cdot (e + f)$$

$$r = P_5 + P_4 - P_2 + P_6$$

$$s = P_1 + P_2$$

$$t = P_3 + P_4$$

$$u = P_5 + P_1 - P_3 - P_7$$

7 mults, 18 adds/subs.

Note: No reliance on commutativity of mult!

应用6: Strassen's矩阵乘法

- Multiply 2×2 matrices with only 7 recursive mults.

$$P_1 = a \cdot (f - h)$$

$$P_2 = (a + b) \cdot h$$

$$P_3 = (c + d) \cdot e$$

$$P_4 = d \cdot (g - e)$$

$$P_5 = (a + d) \cdot (e + h)$$

$$P_6 = (b - d) \cdot (g + h)$$

$$P_7 = (a - c) \cdot (e + f)$$

$$r = P_5 + P_4 - P_2 + P_6$$

$$= (a + d)(e + h)$$

$$+ d(g - e) - (a + b)h$$

$$+ (b - d)(g + h)$$

$$= ae + ah + de + dh$$

$$+ dg - de - ah - bh$$

$$+ bg + bh - dg - dh$$

$$= ae + bg$$

应用6: Strassen's算法复杂度

Strassen's algorithm

- 1. Divide:** Partition A and B into $(n/2) \times (n/2)$ submatrices. Form terms to be multiplied using $+$ and $-$.
- 2. Conquer:** Perform 7 multiplications of $(n/2) \times (n/2)$ submatrices recursively.
- 3. Combine:** Form C using $+$ and $-$ on $(n/2) \times (n/2)$ submatrices.

$$T(n) = 7 T(n/2) + \Theta(n^2)$$

应用6: Strassen's算法复杂度

Analysis of Strassen

$$T(n) = 7 T(n/2) + \Theta(n^2)$$

$$n^{\log_b a} = n^{\log_2 7} \approx n^{2.81} \Rightarrow \text{CASE 1} \Rightarrow T(n) = \Theta(n^{\lg 7}).$$

The number 2.81 may not seem much smaller than 3, but because the difference is in the exponent, the impact on running time is significant. In fact, Strassen's algorithm beats the ordinary algorithm on today's machines for $n \geq 30$ or so.

应用6：矩阵乘法问题算法进展

- 作为众多GPU算子的基础操作，是高性能计算的重要问题之一，也是AI等应用的基石。它被融入到人们每天使用的大部分算法中，用于各种任务，从显示更清晰的计算机图形到解决网络理论中的物流问题。就像在计算机的其它领域一样，速度至关重要。即使是微小的改进，最终也可能大大减少算法所需要的时间、计算能力和金钱。
 - 它的算法机制本身相当简单，但为了达到更快的速度，人们多年来不懈努力，尽可能优化算法的时间复杂度，并感兴趣弄清这个过程到底有多快。
 - 在Strassen之后又有许多算法改进了矩阵乘法的计算时间复杂性。
 - 1981年： $\Theta(n^{2.494})$
 - 1987年： $\Theta(n^{2.376})$ (Don Coppersmith, Shmuel Winograd, Matrix Multiplication via Arithmetic Progressions, STOC 1987: 1-6).
-

应用6：矩阵乘法问题算法进展

- 在1990年后，相关研究进入了长达20年的冬眠期。直到爱丁堡大学数学系博士生 Andrew Stothers 在2010年于其博士论文里面，提出了一个新的算法，使时间复杂度进一步降维 ($O(n^{2.374})$)
 - 2011年底，斯坦福大学的 Virginia Vassilevska Williams 在基于 Andrew Stothers 的基础上，把时间复杂度降至 $O(n^{2.3728642})$ 。
 - 2014年秋由名古屋大学 François Le Gall 简化的斯坦福方法，时间复杂度达到 $O(n^{2.3728639})$ 。
 - 2017 文献综述提到 “The exponent has still not passed through the barrier of 2.37286”
 - 2020 年 10 月，来自哈佛大学与 MIT 的两位研究者发表了一篇论文，他们创建了有史以来矩阵相乘的最快算法，复杂度的结果小于 $n^{2.37286}$ ，相比于之前最快算法，计算复杂度下降了 10 万分之一。其中，论文一作 Josh Alman 是哈佛大学的博士后研究生，主要研究算法设计与复杂度理论。二作 Vassilevska Williams 是 MIT 计算机科学与人工智能实验室 (CSAIL) 副教授，致力于将组合和图论工具应用于计算领域。
-

应用6：矩阵乘法问题算法进展

- ❑ 2024年在《量子杂志》的一篇报道中，提到推动矩阵乘法速度进一步提升的两篇论文，其中清华大学姚班的一位大四本科生全程参与了两篇论文的撰写，为该领域的算法改进带来了全新的希望。
 - ❑ 论文1: Ran Duan, Hongxun Wu, Renfei Zhou, Faster Matrix Multiplication via Asymmetric Hashing, FOCS, 2023
 - ❑ 来自清华大学的段然，周任飞和加州大学伯克利分校的Hongxun Wu，在解决这个长期存在的问题上迈出了重要的一步，撰写的论文长达87页，对于三位研究者的成果，Le Gall表示尽管改进本身相对较小，但是从概念上讲比以往改进都大。
 - ❑ 论文2: Virginia Vassilevska Williams, Yinzhao Xu, Zixuan Xu, Renfei Zhou, New bounds for Matrix Multiplication: from Alpha to Omega, ArXiv.abs/2307.07970
 - ❑ 哈佛大学理论计算机科学家 William Kuszmaul 评价“这是一项重大的技术突破，是十多年来我们看到的矩阵乘法的最大改进”。
-

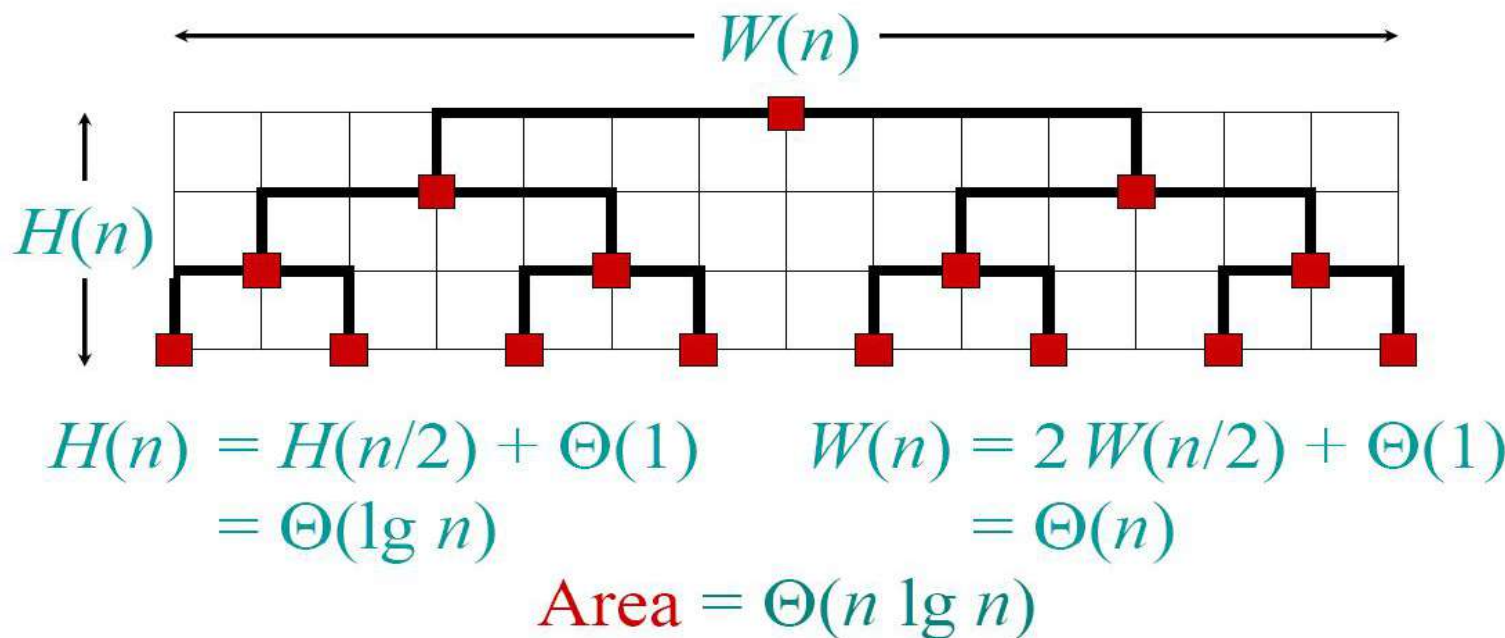
应用6：矩阵乘法问题算法进展

□ 是否能找到 $O(n^2)$ 的算法？ 目前为止还没有结果。

应用7：超大规模集成电路版图设计

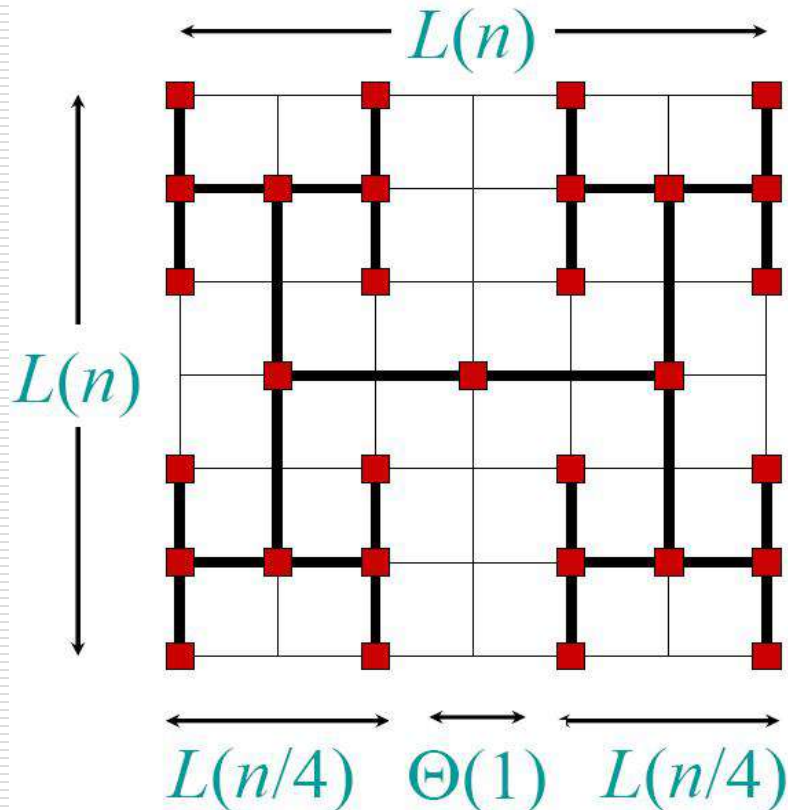
VLSI layout

Problem: Embed a complete binary tree with n leaves in a grid using minimal area.



应用7：超大规模集成电路版图设计

H-tree embedding



$$\begin{aligned} L(n) &= 2L(n/4) + \Theta(1) \\ &= \Theta(\sqrt{n}) \end{aligned}$$

$$\text{Area} = \Theta(n)$$

应用8：残缺棋盘覆盖问题

□ 残缺棋盘：

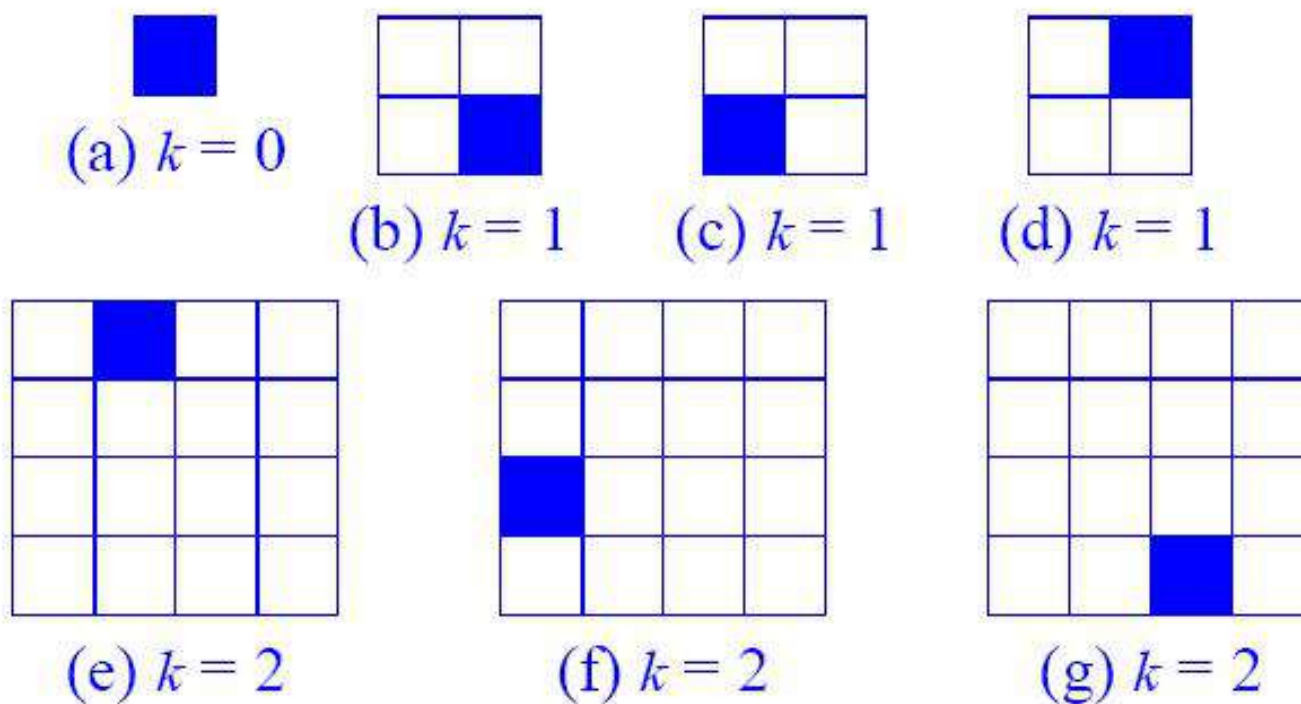
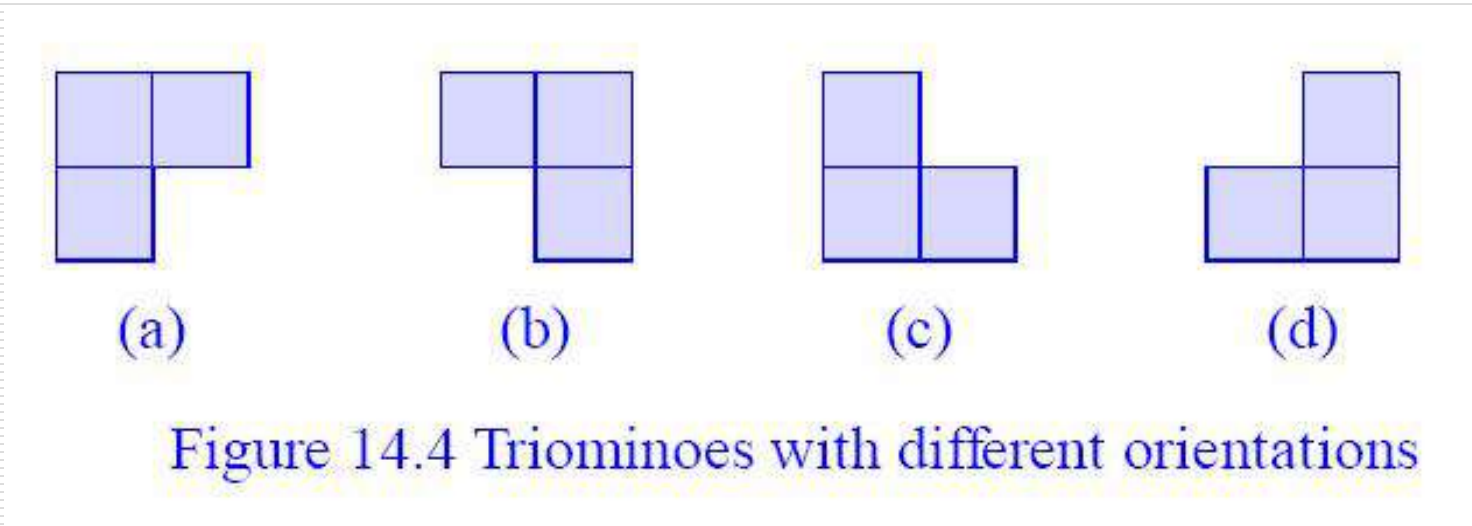


Figure 14.3 Example defective chessboards

应用8：残缺棋盘覆盖问题

□ 不同方向的三格板

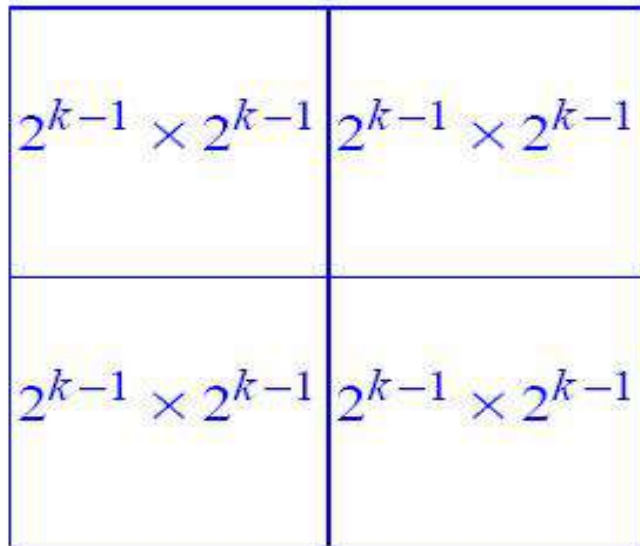


□ 残缺棋盘的问题要求用三格板 (triominoes) 覆盖残缺棋盘。

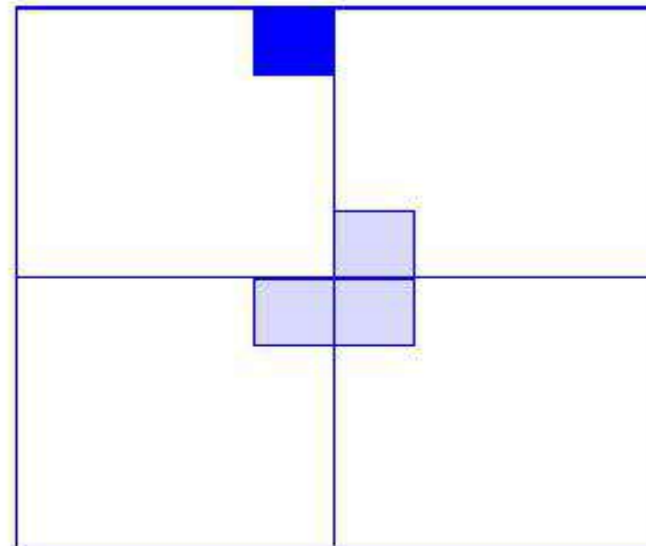
- 在此覆盖中，两个三格板不能重叠，三格板不能覆盖残缺方格，但必须覆盖其他所有的方格

应用8：残缺棋盘覆盖问题

□ 用分治法解决残缺棋盘覆盖问题



(a) Partitioning



(b) Triomino placed

Figure 14.5 Partitioning a $2^k \times 2^k$ board

应用8：残缺棋盘覆盖问题-分治算法

- 将 $2^k \times 2^k$ 的棋盘，先分成相等的四块子棋盘，其中残缺方格位于四个中的一个，构造剩下没残缺方格三个子棋盘，将它们中的一个方格也假设为残缺方格。
 - 左上的子棋盘（若不存在残缺方格）----则将该子棋盘右下角的那个方格假设为残缺方格
 - 右上的子棋盘（若不存在残缺方格）----则将该子棋盘左下角的那个方格假设为残缺方格
 - 左下的子棋盘（若不存在残缺方格）----则将该子棋盘右上角的那个方格假设为残缺方格
 - 右下的子棋盘（若不存在残缺方格）----则将该子棋盘左上角的那个方格假设为残缺方格
 - 上面四种，只可能且必定只有三个成立，那三个假设的残缺方格刚好构成一个三格板，可以给它们作上相同的标记。这样四个子棋盘就分别都和原来的大棋盘类似，从而可以用递归算法解决。
-

应用8：残缺棋盘覆盖问题-部分代码

```
void TileBoard(int tr, int tc, int dr, int dc, int size)
```

```
{// 覆盖残缺棋盘
```

```
    if (size == 1) return;
```

```
        int t = tile++, // 所使用的三格板的数目
```

```
        s = size/2; // 象限大小
```

```
// 覆盖左上象限
```

```
    if (dr < tr + s && dc < tc + s)
```

```
        // 残缺方格位于本象限
```

```
        TileBoard ( t r, tc, dr, dc, s);
```

```
    else {// 本象限中没有残缺方格
```

```
        //则将该子棋盘右下角的那个方格假设为残缺方格
```

```
        Board[tr + s - 1][tc + s - 1] = t;
```

```
        // 覆盖其余部分
```

```
        TileBoard ( t r, tc, tr+s-1, tc+s-1, s);} 
```

```
// 覆盖右上象限
```

```
    if (dr < tr + s && dc >= tc + s)
```

```
        // 残缺方格位于本象限
```

```
        TileBoard ( t r, tc+s, dr, dc, s);
```

```
    else {// 本象限中没有残缺方格
```

```
        //则将该子棋盘左下角的那个方格假设为残缺方格
```

```
        Board[tr + s - 1][tc + s] = t;
```

```
        // 覆盖其余部分
```

应用8：残缺棋盘覆盖问题-部分代码

```
    TileBoard( tr, tc+s, tr+s-1, tc+s, s);}
// 覆盖左下象限
    if (dr >= tr + s && dc < tc + s)           // 残缺方格位于本象限
        TileBoard(tr+s, tc, dr, dc, s);
    else { // 则将该子棋盘右上角的那个方格假设为残缺方格
        Board[tr + s][tc + s - 1] = t;         // 覆盖其余部分
        TileBoard(tr+s, tc, tr+s, tc+s-1, s);}
// 覆盖右下象限
    if (dr >= tr + s && dc >= tc + s)           // 残缺方格位于本象限
        TileBoard(tr+s, tc+s, dr, dc, s);
    else { // 则将该子棋盘左上角的那个方格假设为残缺方格
        Board[tr + s][tc + s] = t;             // 覆盖其余部分
        TileBoard(tr+s, tc+s, tr+s, tc+s, s);}
}
void OutputBoard(int size)
{
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++)
            cout << setw (5) << Board[i][j];
        cout << endl;
    }
}
```

应用8：残缺棋盘覆盖问题-复杂度

- 假设 $n=4^k$
 - 需 $(n-1)/3$ 个三格板填满棋盘（可证明整除）
 - 算法的时间复杂度：
$$t(n)=4t(n/4)+c \quad \text{if } k>1$$
$$t(n)=1 \quad \text{if } k=1,$$
 - 使用Master方法求解， $\log_b a=1$, case 1: $\epsilon=0.5$
 - 算法复杂度： $t(n)=\Theta(n)$
 - 是否还有其它分析方法？
-

应用9：归并排序（merge sort）

- 问题：给定含 n 个元素的数据集合，对给定的 n 个元素进行排序。
 - 可采用平衡分割法来分割 n 个元素,即将 n 个元素分为 A 和 B 两个集合,其中 A 集合中含有 n/k 个元素, B 集合中包含其余的元素。
 - 然后递归地使用分治法对 A 和 B 进行排序。
 - 当 A 或 B 内元素数 $< k$ 时使用插入排序。
 - 然后采用一个称为归并（merge）的过程，将已排好序的 A 和 B 合并成一个集合。
-

应用9：归并排序-例题

- 考虑8个元素, 值分别为[10, 4, 6, 3, 8, 2, 5, 7]。
 - 1) $k=2$ 时排序;
 - $A1=[10,4, 6, 3]$ $A2=[8,2,5,7]$
 - $A11=[10,4], A12=[6, 3], A21=[8,2], A22=[5,7]$
 - $A111=[10], A112=[4], A121=[6], A122=[3],$
 $A211=[8], A212=[2], A221=[5], A222=[7]$
 - 2) $k=4$ 时排序, 如何操作?
-

应用9：归并排序算法伪代码

```
template<class T>
void sort( T E, int n)
{ //对E中的n 个元素进行排序，k为全局变量
    if (n >= k) {
        i = n/k;
        j = n-i;
        令A 包含E中的前 i 个元素
        令 B 包含E中余下的 j 个元素
        sort(A,i);
        sort(B,j);
        merge(A,B,E,i,j); //把A 和 B 合并到 E
    }
    else 使用插入排序算法对 E 进行排序
}
```

应用9：归并排序算法复杂度

- 设 $t(n)$ 为归并排序算法复杂度函数，则有以下递推公式：

$$t(n) = \begin{cases} d & n < k \\ t(n/k) + t(n - n/k) + cn & n \geq k \end{cases}$$

- 当 $n/k \approx n - (n/k)$ 时 $t(n)$ 的值最小（最小平衡原理）
- 当 $k=2$ 时,分治法通常具有最佳性能:当 $k>2$ 时递归展开的深度 $\log_a n$, $a=k/(k-1)$,超过 $\log_2 n$.
- 当 $k=2$ 时 ,可得到如下递推公式:

$$t(n) = \begin{cases} d & n \leq 1 \\ t(\lfloor n/2 \rfloor) + t(\lceil n/2 \rceil) + cn & n > 1 \end{cases}$$

应用10：快速排序（quick sort）

- 分治法还可以用于实现另一种完全不同的排序方法：

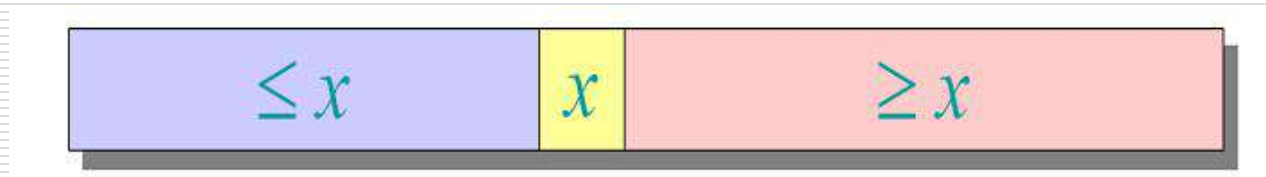
快速排序

- 在这种方法中, n 个元素被分成三段:左段left, 右段right和中段middle.
 - 中段仅包含一个元素;左段中各元素都小于等于中段元素;右段中各元素都大于等于中段元素.
 - 左段和右段中的元素可以独立排序,并且不必对左段和右段的排序结果进行合并, middle中的元素被称为支点(pivot).
-

应用10：快速排序-算法

- 对n个元素进行快速排序

1. **分**：将n个元素在支点x分成两个子序列，支点左边元素都小于等于 支点元素，右边元素都大于等于支点元素



2. **治**：递归的对两个字序列使用快速排序算法
3. **归并**：将已排好序的子序列归并

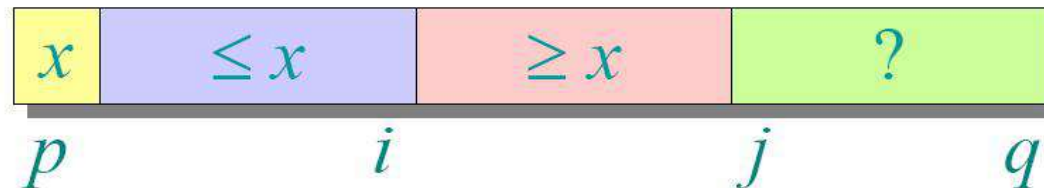
应用10：快速排序算法-如何分

Partitioning subroutine

```
PARTITION( $A, p, q$ )  $\triangleright A[p \dots q]$   
   $x \leftarrow A[p]$   $\triangleright$  pivot =  $A[p]$   
   $i \leftarrow p$   
  for  $j \leftarrow p + 1$  to  $q$   
    do if  $A[j] \leq x$   
      then  $i \leftarrow i + 1$   
           exchange  $A[i] \leftrightarrow A[j]$   
  exchange  $A[p] \leftrightarrow A[i]$   
  return  $i$ 
```

Running time
= $O(n)$ for n
elements.

Invariant:



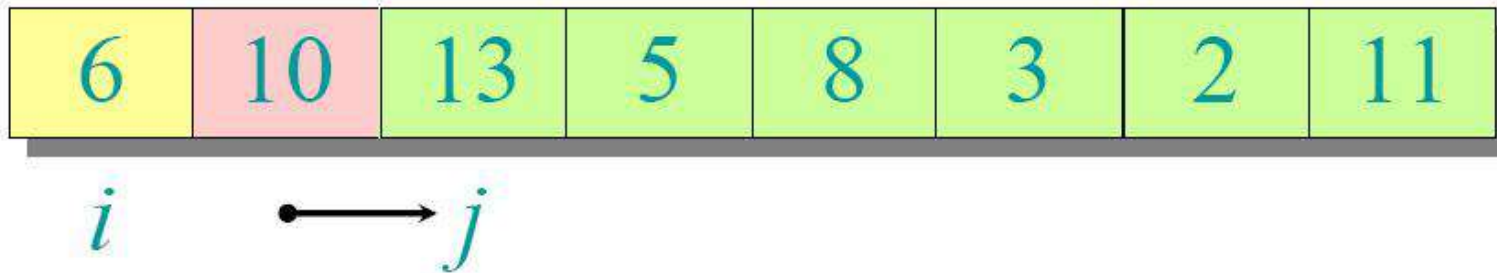
应用10：快速排序算法-分法举例

Example of partitioning

6	10	13	5	8	3	2	11
i	j						

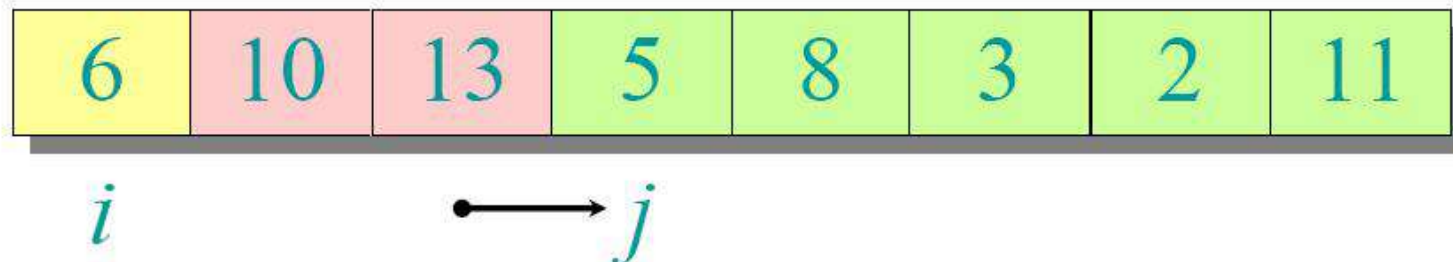
应用10：快速排序算法-分法举例

Example of partitioning



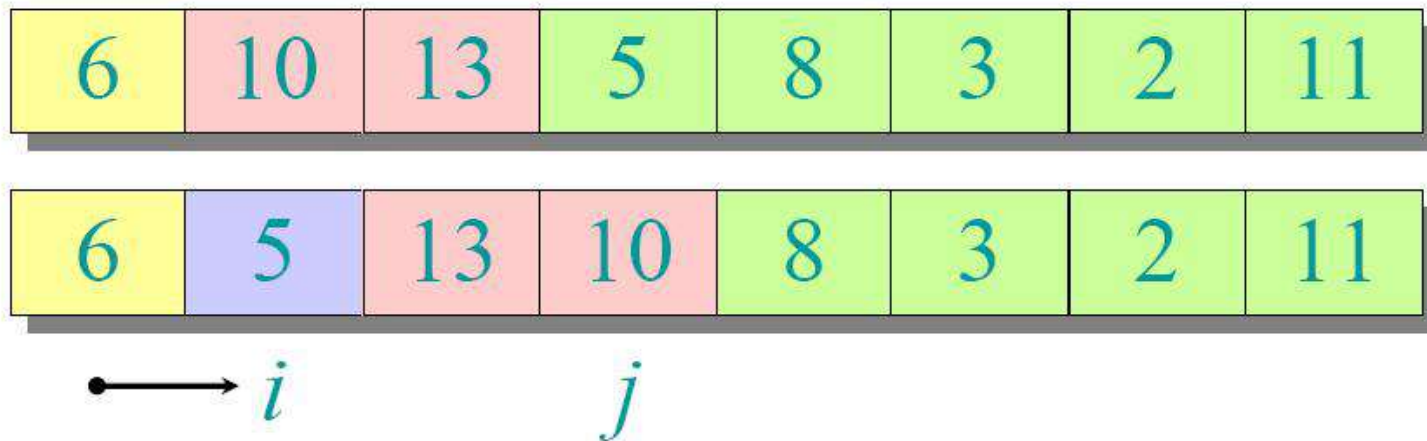
应用10：快速排序算法-分法举例

Example of partitioning



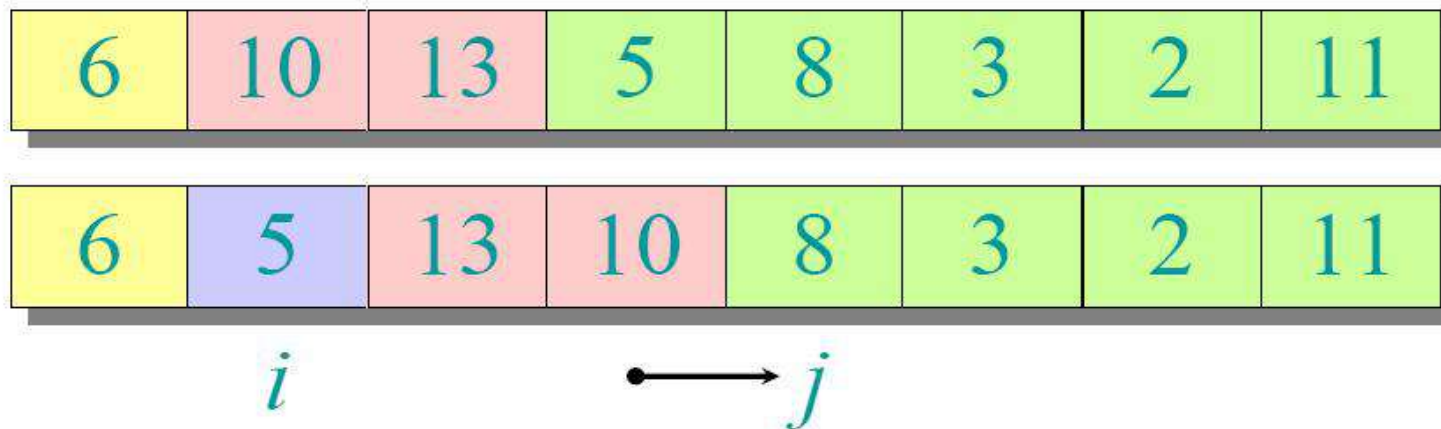
应用10：快速排序算法-分法举例

Example of partitioning



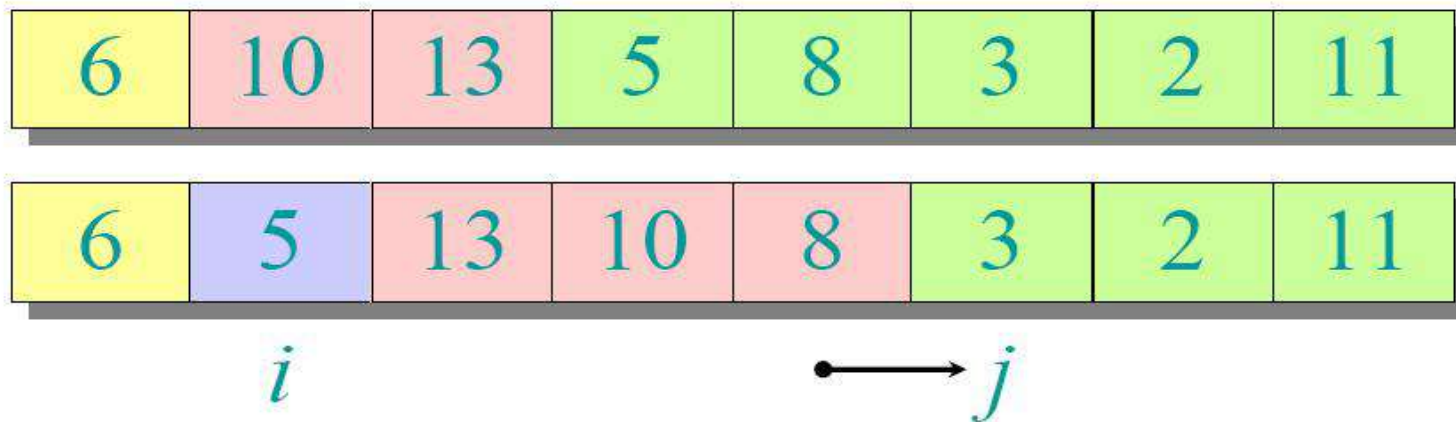
应用10：快速排序算法-分法举例

Example of partitioning



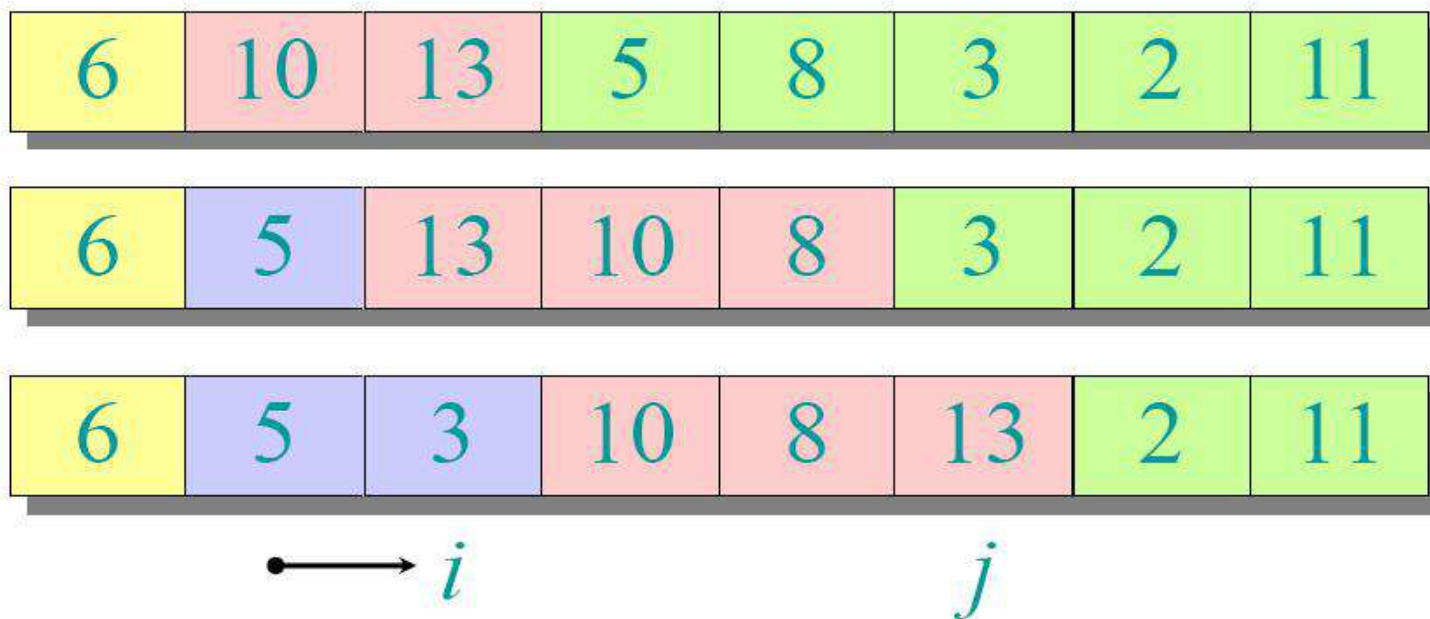
应用10：快速排序算法-分法举例

Example of partitioning



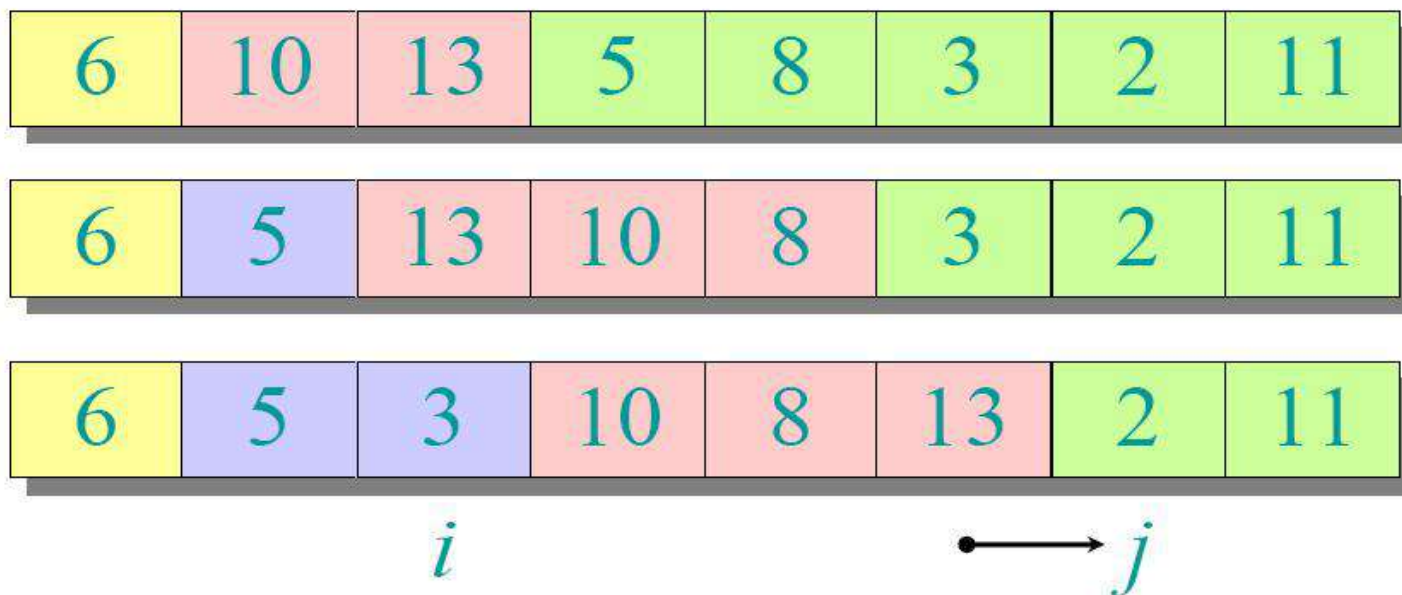
应用10：快速排序算法-分法举例

Example of partitioning



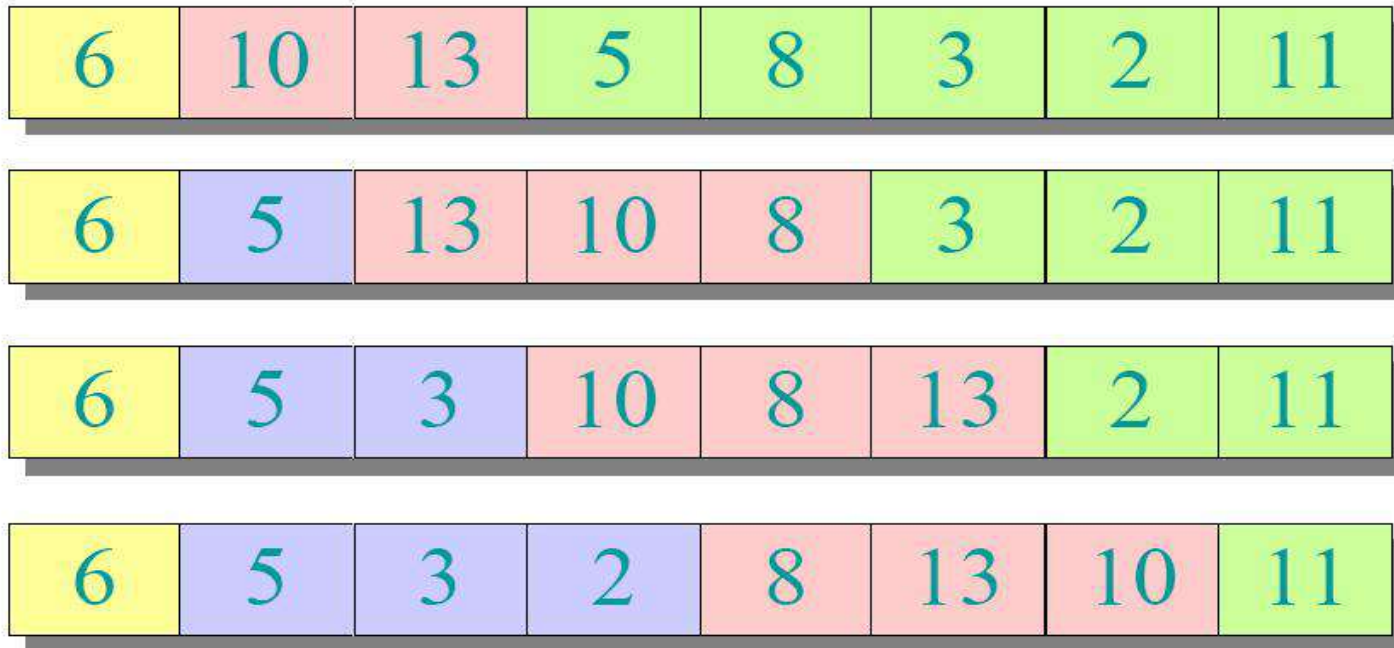
应用10：快速排序算法-分法举例

Example of partitioning



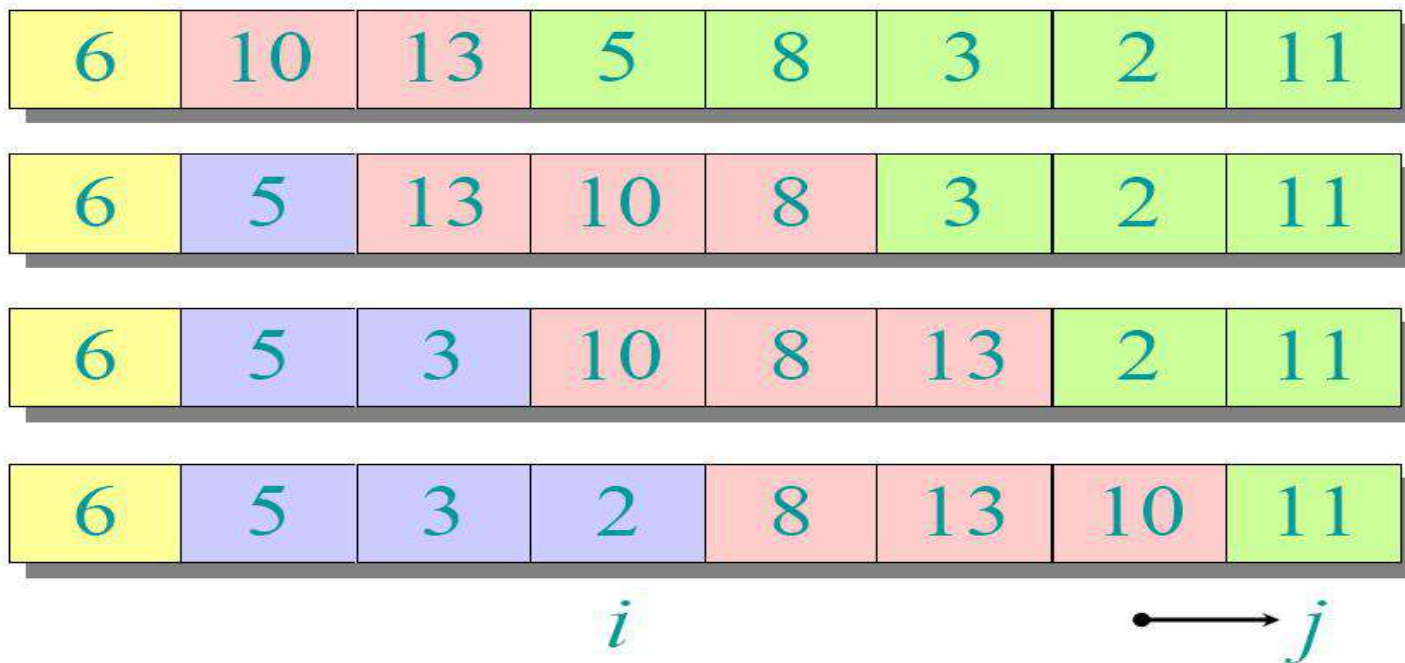
应用10：快速排序算法-分法举例

Example of partitioning



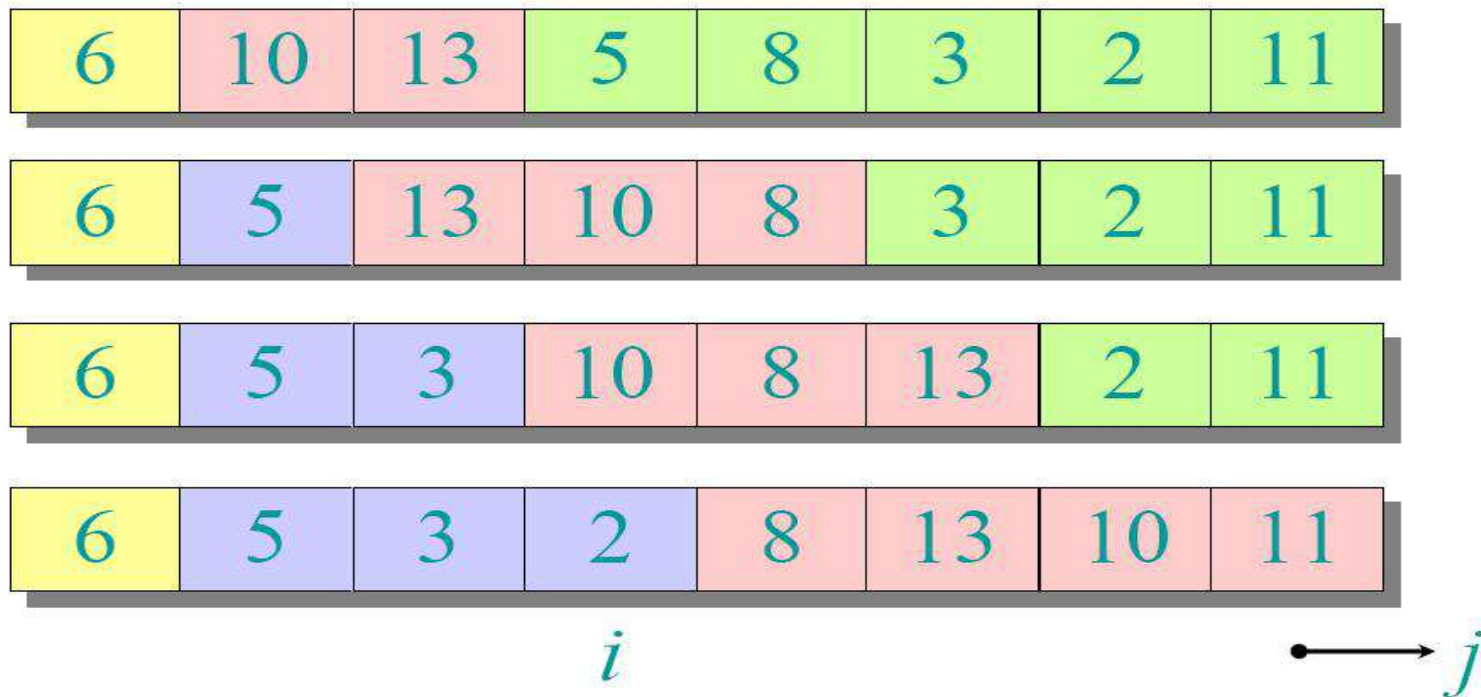
应用10：快速排序算法-分法举例

Example of partitioning



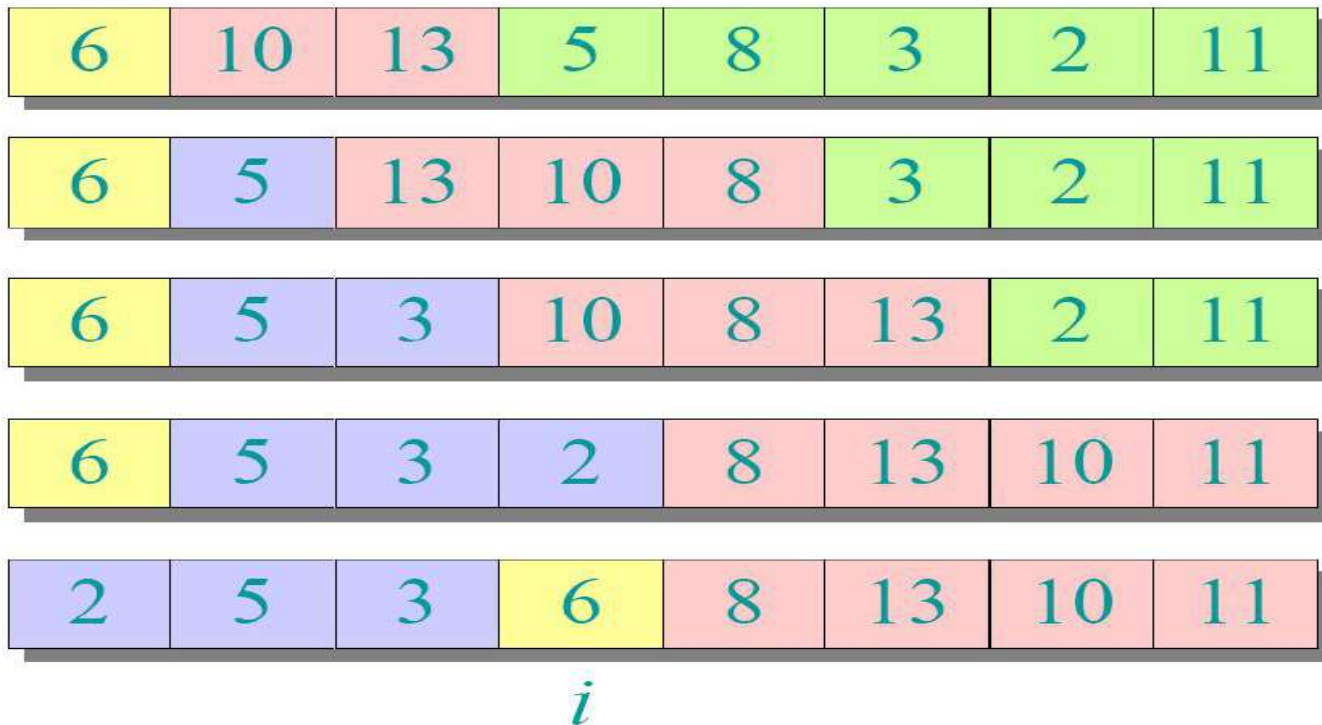
应用10：快速排序算法-分法举例

Example of partitioning



应用10：快速排序算法-分法举例

Example of partitioning



应用10：快速排序算法-伪代码

Pseudocode for quicksort

QUICKSORT(A, p, r)

 if $p < r$

 then $q \leftarrow \text{PARTITION}(A, p, r)$

 QUICKSORT($A, p, q-1$)

 QUICKSORT($A, q+1, r$)

Initial call: QUICKSORT($A, 1, n$)

应用10：快速排序算法-复杂度

Analysis of quicksort

- Assume all input elements are distinct.
 - In practice, there are better partitioning algorithms for when duplicate input elements may exist.
 - Let $T(n)$ = worst-case running time on an array of n elements.
-

应用10：快速排序算法-复杂度

Worst-case of quicksort

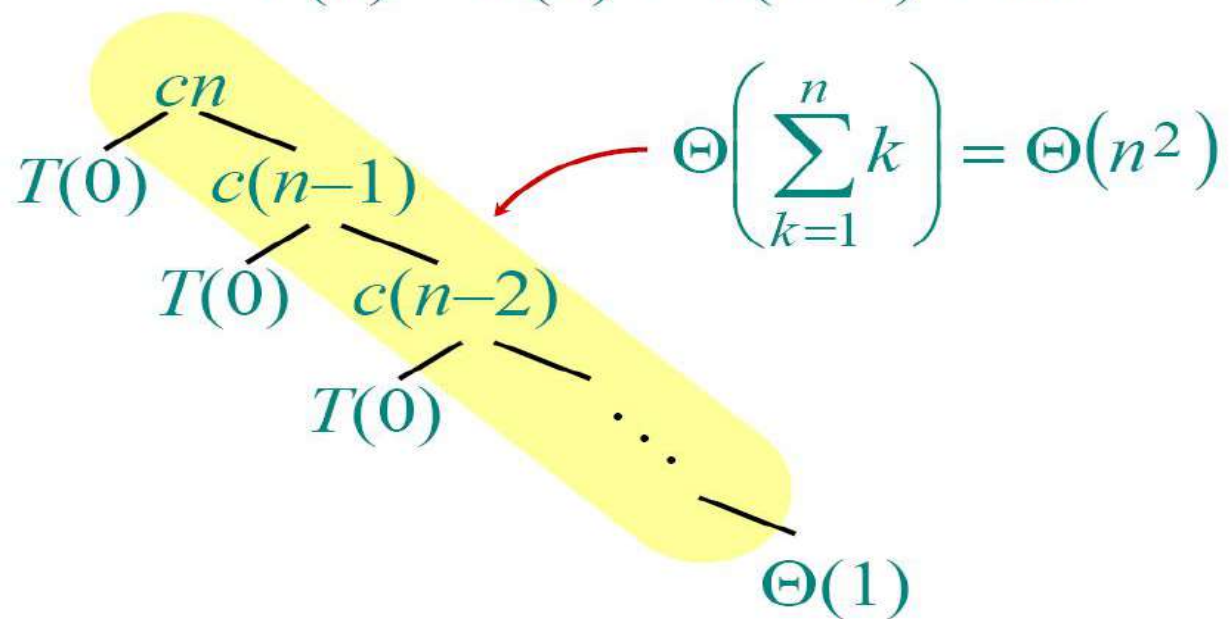
- Input sorted or reverse sorted.
- Partition around min or max element.
- One side of partition always has no elements.

$$\begin{aligned}T(n) &= T(0) + T(n-1) + \Theta(n) \\&= \Theta(1) + T(n-1) + \Theta(n) \\&= T(n-1) + \Theta(n) \\&= \Theta(n^2) \quad (\textit{arithmetic series})\end{aligned}$$

应用10：快速排序算法-复杂度

Worst-case recursion tree

$$T(n) = T(0) + T(n-1) + cn$$



□ 最坏情形时间复杂度： $T(n) = \Theta(n) + \Theta(n^2) = \Theta(n^2)$

应用10：快速排序算法-复杂度

Best-case analysis

(For intuition only!)

If we're lucky, PARTITION splits the array evenly:

$$\begin{aligned} T(n) &= 2T(n/2) + \Theta(n) \\ &= \Theta(n \lg n) \quad (\text{same as merge sort}) \end{aligned}$$

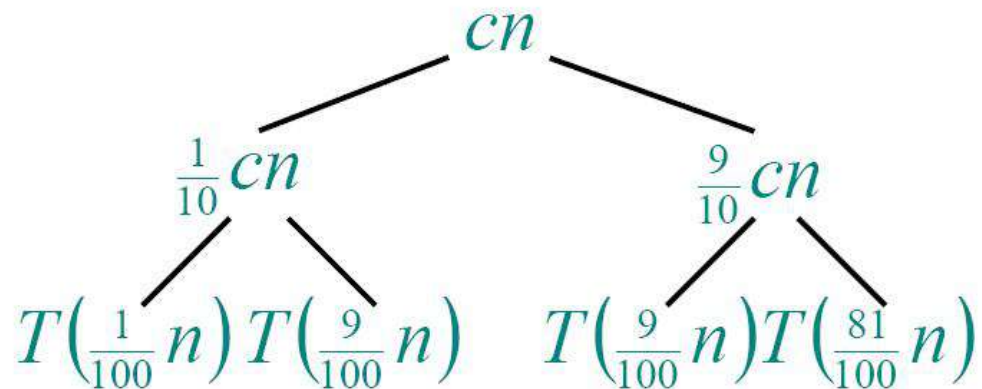
What if the split is always $\frac{1}{10} : \frac{9}{10}$?

$$T(n) = T\left(\frac{1}{10}n\right) + T\left(\frac{9}{10}n\right) + \Theta(n)$$

What is the solution to this recurrence?

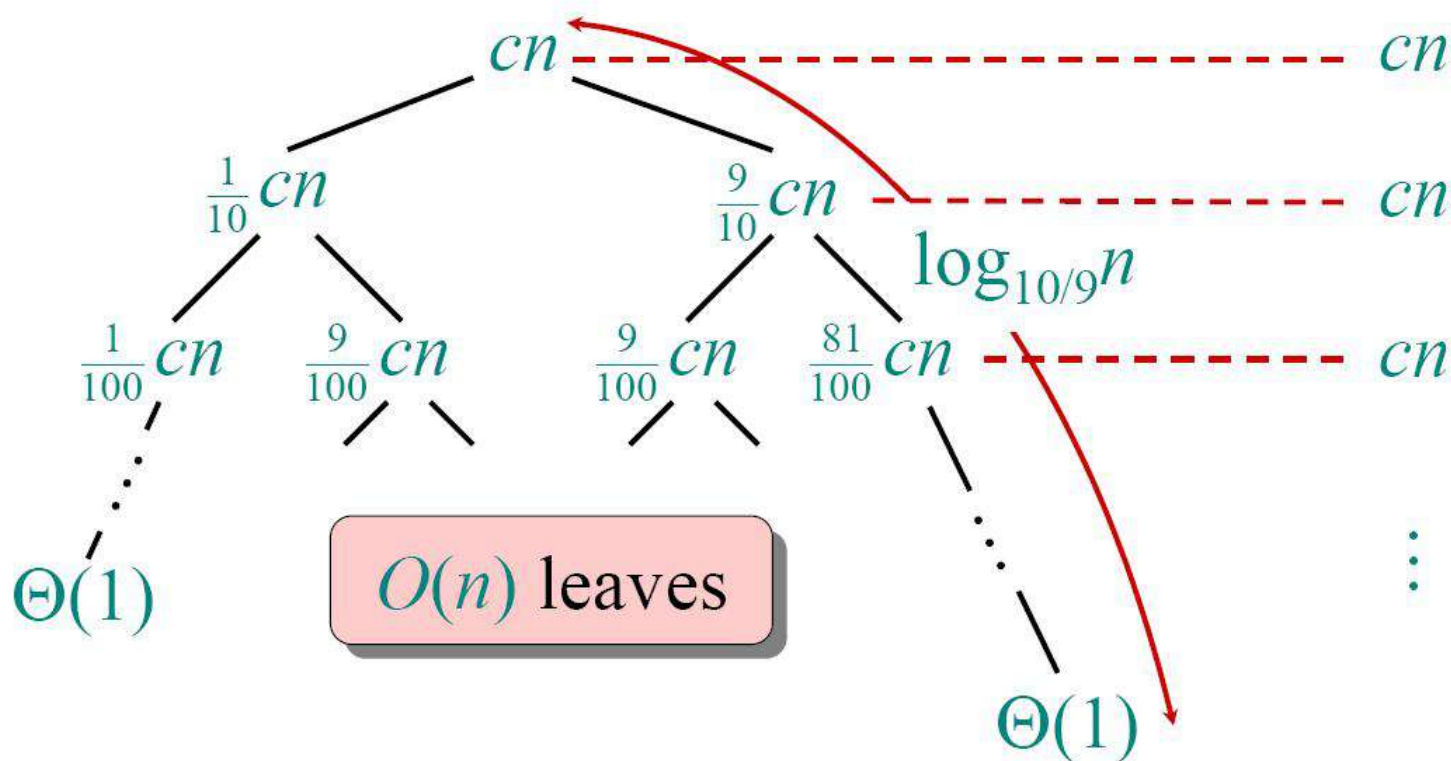
应用10：快速排序算法-复杂度

Analysis of “almost-best” case



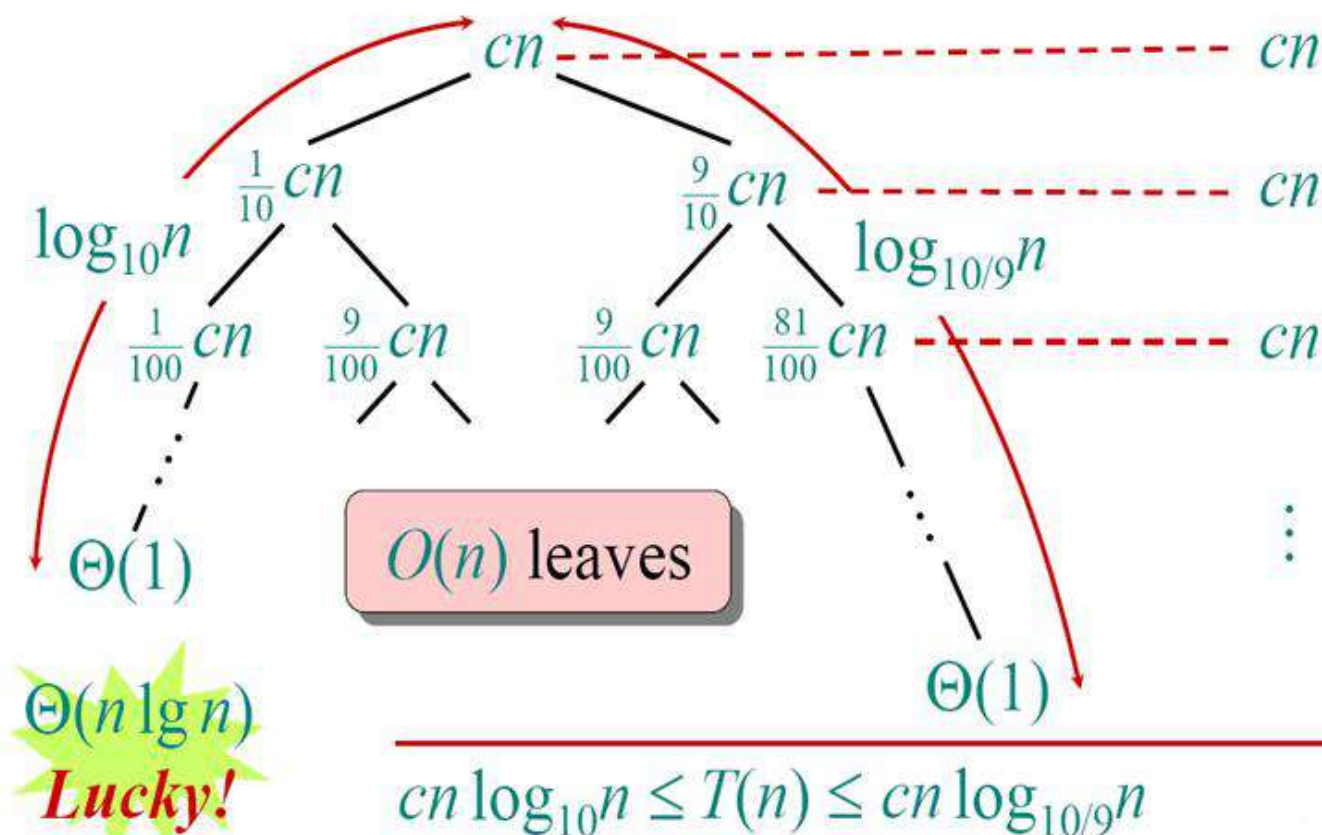
应用10：快速排序算法-复杂度

Analysis of “almost-best” case



应用10：快速排序算法-复杂度

Analysis of “almost-best” case



应用10：快速排序算法-复杂度

□ 平均情形时间复杂度：

- 设 $T(n) \leq d \quad n \leq 1$

- 当 $n > 1$ 时

$$T(n) = cn + \frac{1}{n} \sum_{s=0}^{n-1} [T(s) + T(n-s-1)]$$

- 假定关键字彼此不同则平均关键字比较次数

$$T(n) \leq n + 1 + \frac{1}{n} \sum_{s=0}^{n-1} \{T(s) + T(n-s-1)\}$$

应用10：快速排序算法-复杂度

- 迭代计算如下：

$$nT(n) = n(n+1) + 2\sum_{s=0}^{n-1} T(s)$$

$$(n+1)T(n+1) = (n+1)(n+2) + 2\sum_{s=0}^n T(s)$$

$$(n+1)T(n+1) - nT(n) = 2(n+1) + 2T(n) \Rightarrow$$

$$(n+1)T(n+1) = (n+2)T(n) + 2(n+1)$$

$$\frac{T(n+1)}{n+2} = \frac{T(n)}{n+1} + \frac{2}{n+2}$$

应用10：快速排序算法-复杂度

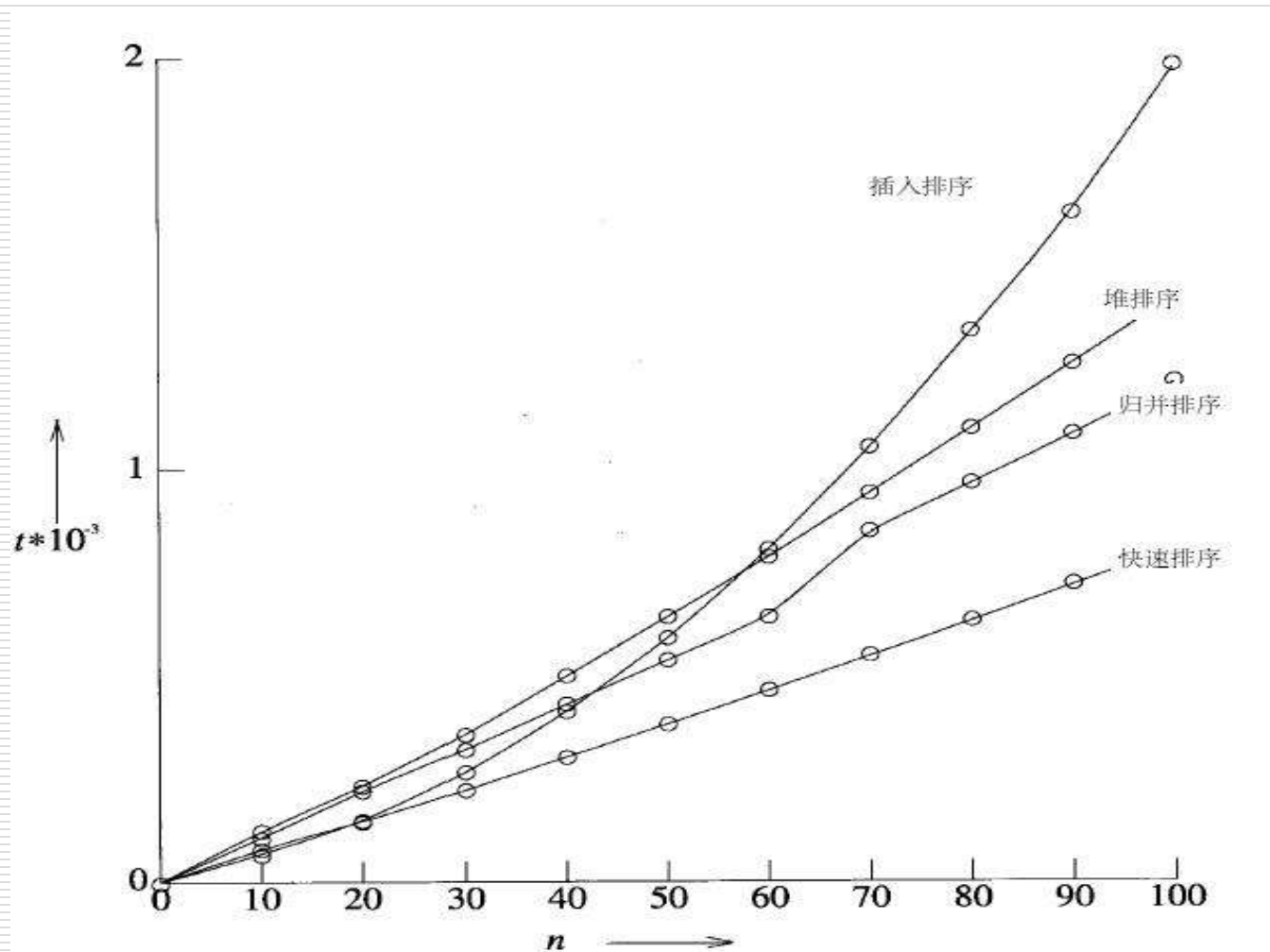
- 令 $f(n) = T(n) / (n+1)$, 有:
- $f(n) = f(n-1) + 2 / (n+1)$
- 迭代展开, 则 $f(n) = f(1) + \sum (2/n)$ 求和从 $n=2$ 开始.
- 调和级数收敛到 $\log_e n$, 所以快速排序算法平均情形的复杂度为:

$$T(n) = (n+1) f(n) = \Theta(n \log n)$$

各种排序算法的比较

方法	最坏复杂性	平均复杂性
冒泡排序	n^2	n^2
计数排序	n^2	n^2
插入排序	n^2	n^2
选择排序	n^2	n^2
堆排序	$n \log n$	$n \log n$
归并排序	$n \log n$	$n \log n$
快速排序	n^2	$n \log n$

各排序算法平均时间的曲线图



应用11:选择问题

Select the i th smallest of n elements (the element with *rank* i).

- $i = 1$: *minimum*;
- $i = n$: *maximum*;
- $i = \lfloor (n+1)/2 \rfloor$ or $\lceil (n+1)/2 \rceil$: *median*.

Naive algorithm: Sort and index i th element.

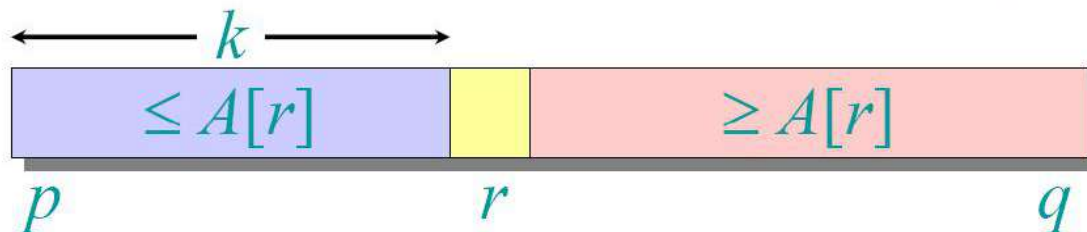
Worst-case running time = $\Theta(n \lg n) + \Theta(1)$
= $\Theta(n \lg n)$,

using merge sort or heapsort (*not* quicksort).

应用11:选择问题-算法伪代码

Randomized divide-and-conquer algorithm

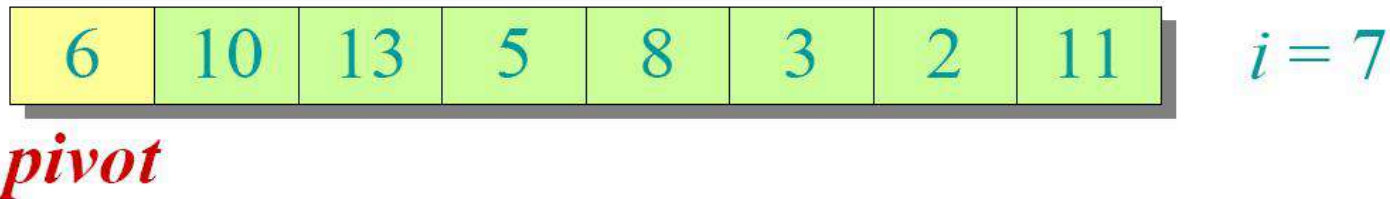
```
RAND-SELECT( $A, p, q, i$ )  $\triangleright$   $i$ th smallest of  $A[p..q]$   
  if  $p = q$  then return  $A[p]$   
   $r \leftarrow$  RAND-PARTITION( $A, p, q$ )  
   $k \leftarrow r - p + 1$   $\triangleright k = \text{rank}(A[r])$   
  if  $i = k$  then return  $A[r]$   
  if  $i < k$   
    then return RAND-SELECT( $A, p, r - 1, i$ )  
  else return RAND-SELECT( $A, r + 1, q, i - k$ )
```



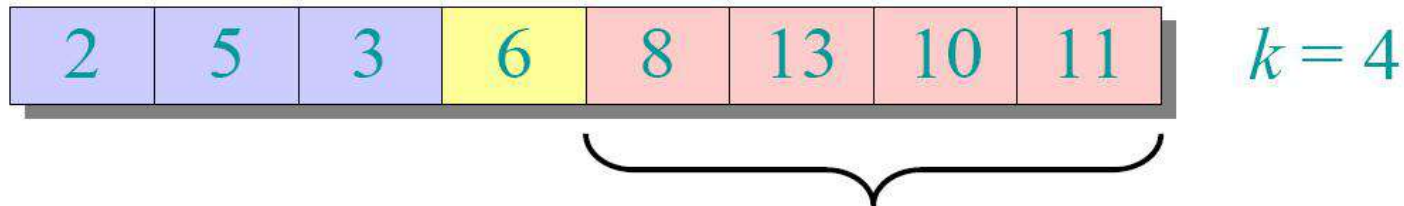
应用11:选择问题-举例

Example

Select the $i = 7$ th smallest:



Partition:



Select the $7 - 4 = 3$ rd smallest recursively.

应用11:选择问题-算法分析

Intuition for analysis

(All our analyses today assume that all elements are distinct.)

Lucky:

$$\begin{aligned} T(n) &= T(9n/10) + \Theta(n) \\ &= \Theta(n) \end{aligned}$$

$$n^{\log_{10/9} 1} = n^0 = 1$$

CASE 3

Unlucky:

$$\begin{aligned} T(n) &= T(n-1) + \Theta(n) \\ &= \Theta(n^2) \end{aligned}$$

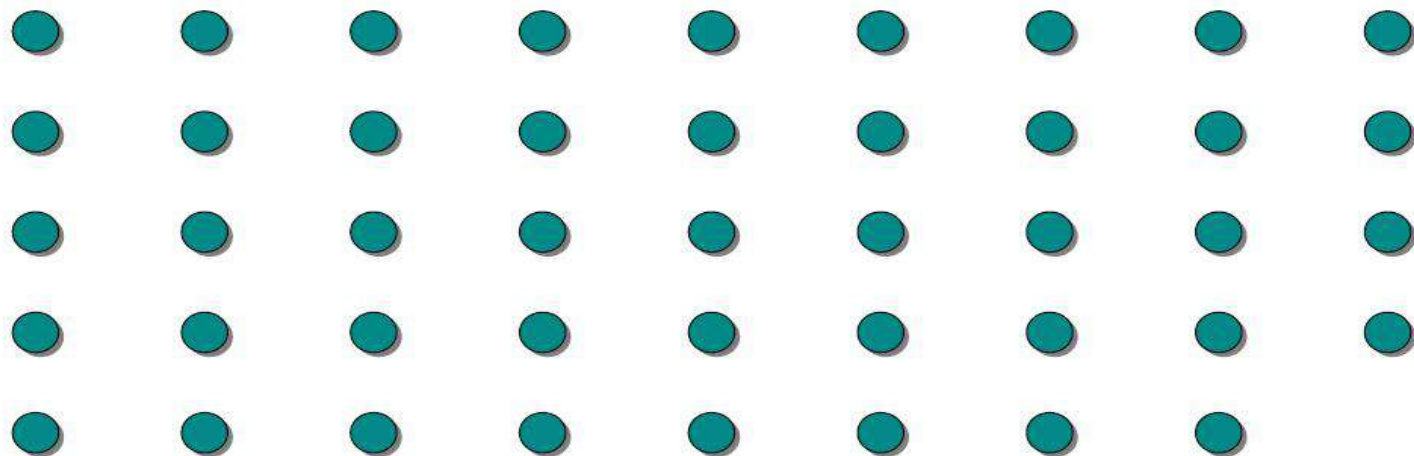
arithmetic series

Worse than sorting!

应用11:选择问题-算法改进

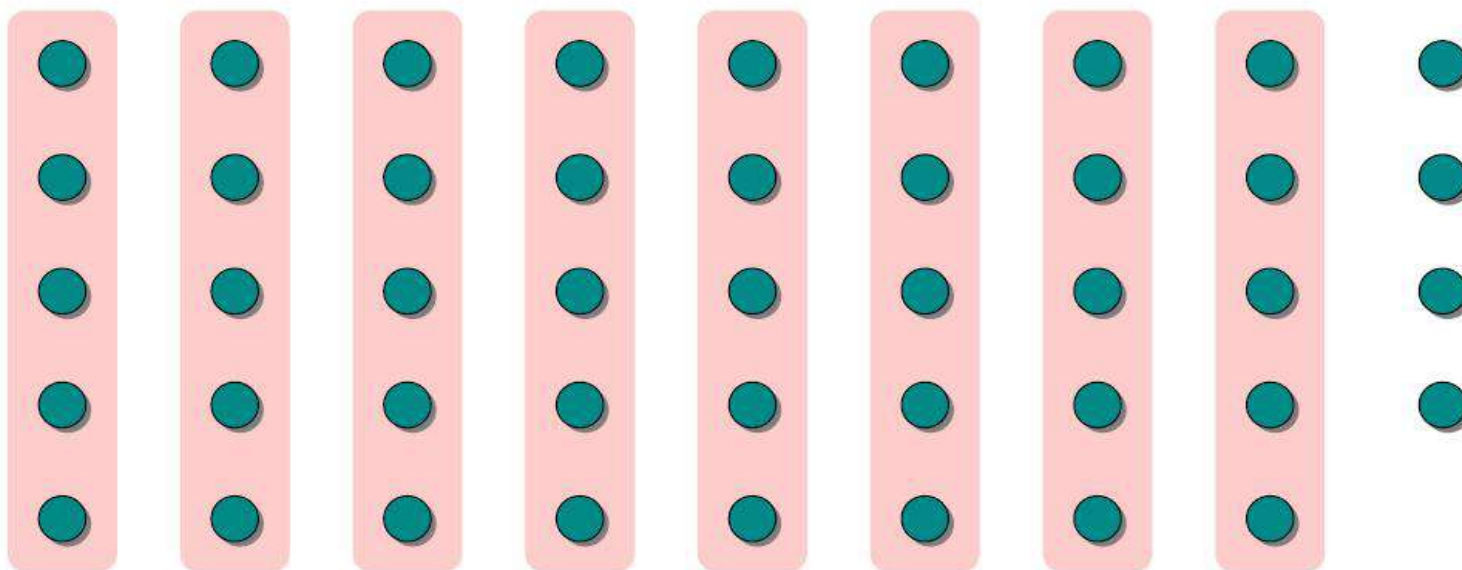
- 改进支点元素选取，从而保证集合划分尽量均衡
- 中间的中间规则

Choosing the pivot



应用11:选择问题-中间的中问规则

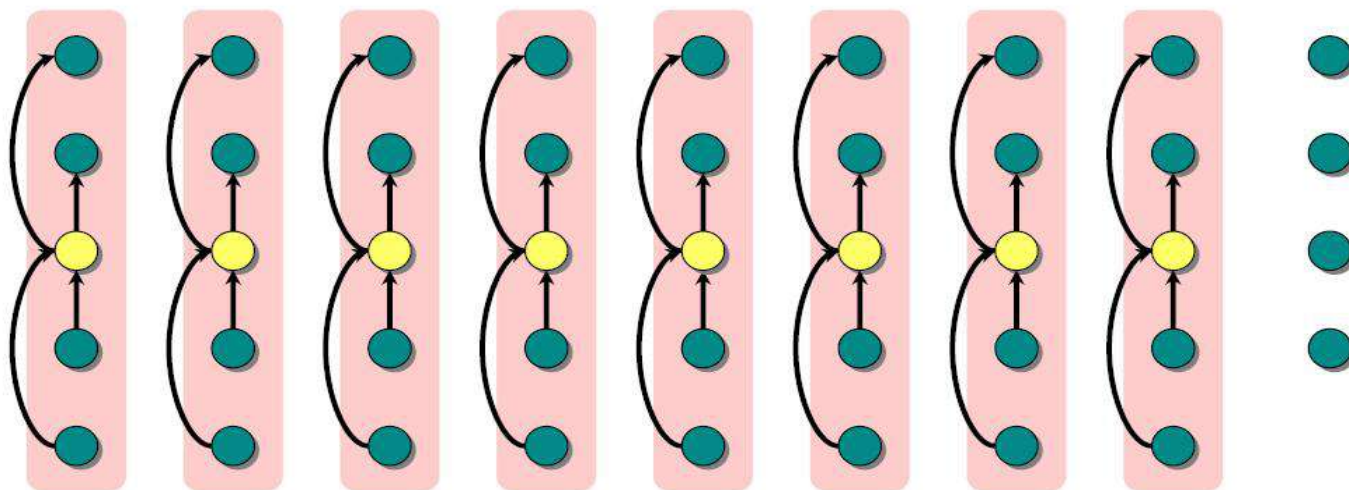
Choosing the pivot



1. Divide the n elements into groups of 5.
-

应用11:选择问题-中间的中问规则

Choosing the pivot



1. Divide the n elements into groups of 5. Find the median of each 5-element group by rote.

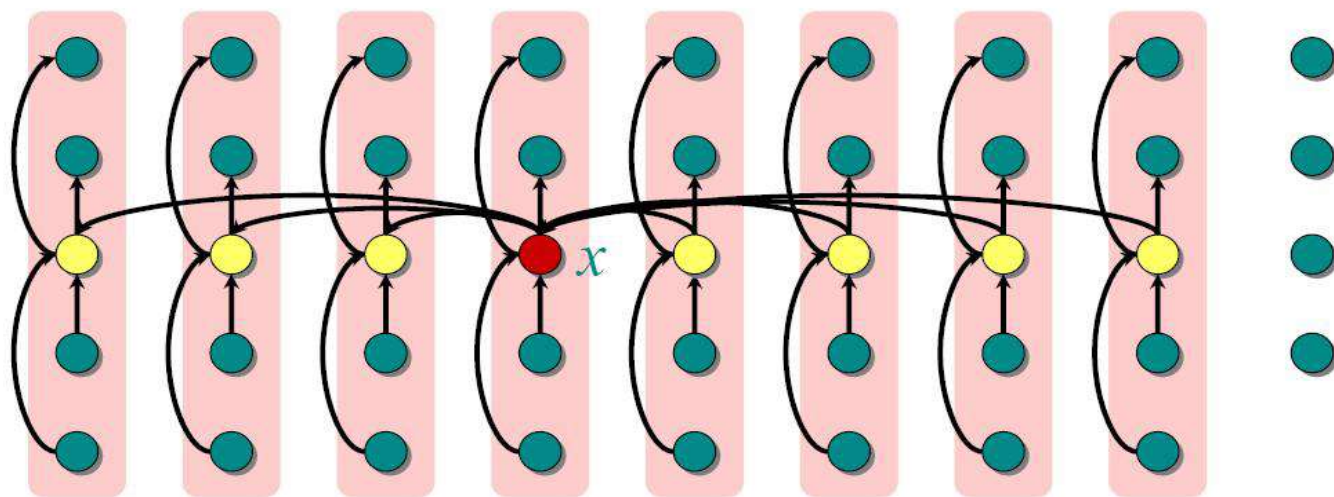
lesser



greater

应用11:选择问题-中间的中问规则

Choosing the pivot



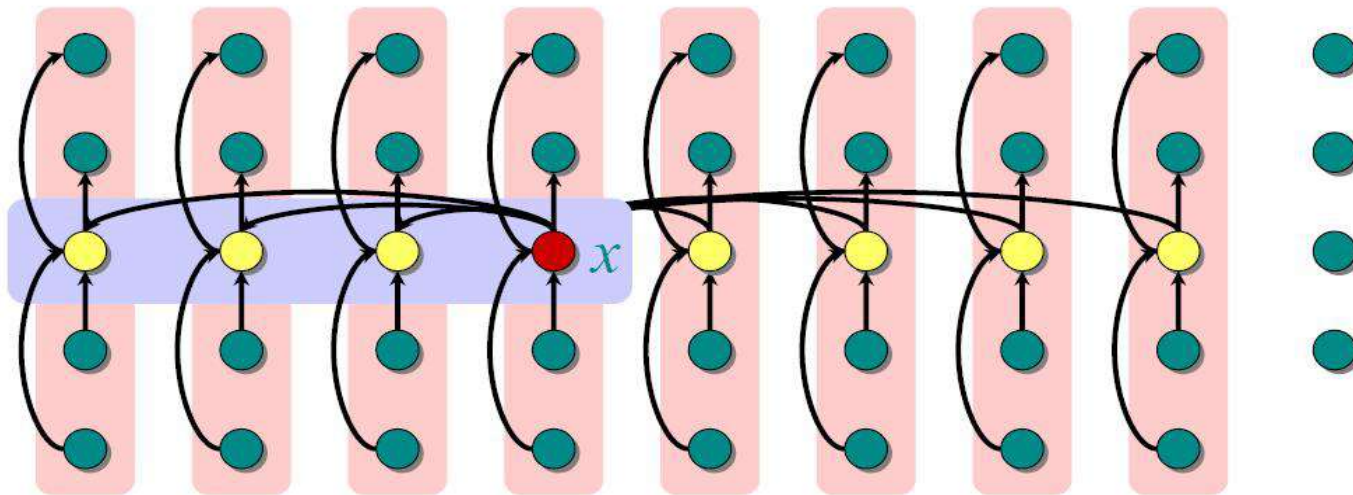
1. Divide the n elements into groups of 5. Find the median of each 5-element group by rote.
2. Recursively SELECT the median x of the $\lfloor n/5 \rfloor$ group medians to be the pivot.

lesser

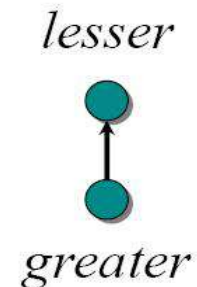
greater

应用11:选择问题-中间的中问规则

Analysis

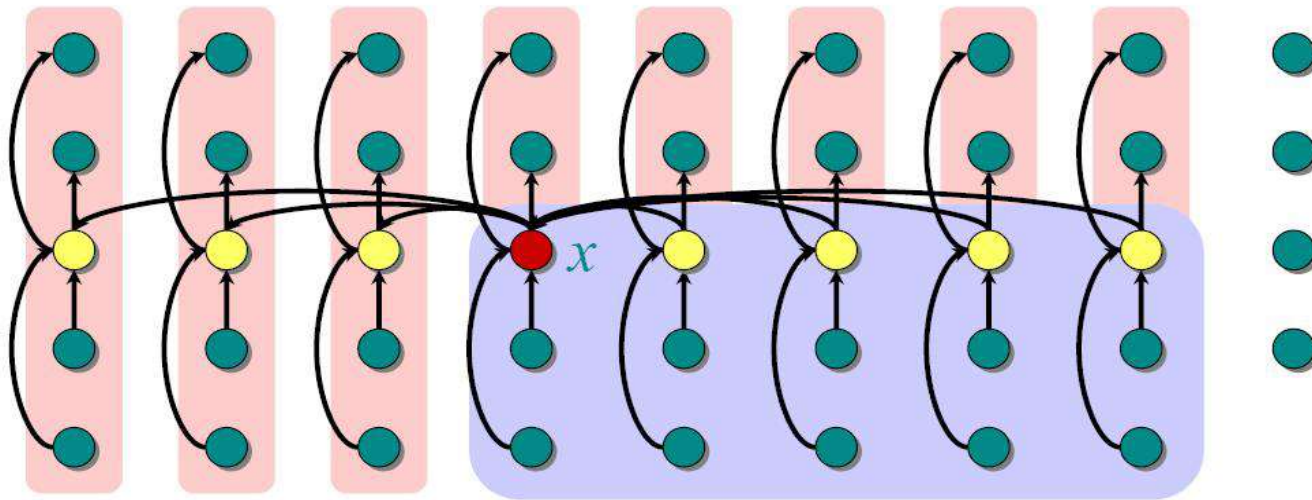


At least half the group medians are $\leq x$, which is at least $\lfloor \lfloor n/5 \rfloor / 2 \rfloor = \lfloor n/10 \rfloor$ group medians.



应用11:选择问题-中间的中问规则

Analysis (Assume all elements are distinct.)



At least half the group medians are $\leq x$, which is at least $\lfloor \lfloor n/5 \rfloor / 2 \rfloor = \lfloor n/10 \rfloor$ group medians.

- Therefore, at least $3\lfloor n/10 \rfloor$ elements are $\leq x$.
- Similarly, at least $3\lfloor n/10 \rfloor$ elements are $\geq x$.

lesser



greater

应用11:选择问题-中间的中问规则

Minor simplification

- For $n \geq 50$, we have $3\lfloor n/10 \rfloor \geq n/4$.
- Therefore, for $n \geq 50$ the recursive call to SELECT in Step 4 is executed recursively on $\leq 3n/4$ elements.
- Thus, the recurrence for running time can assume that Step 4 takes time $T(3n/4)$ in the worst case.
- For $n < 50$, we know that the worst-case time is $T(n) = \Theta(1)$.

应用11:选择问题-算法改进

Developing the recurrence

$T(n)$	SELECT(i, n)
$\Theta(n)$	{ 1. Divide the n elements into groups of 5. Find the median of each 5-element group by rote.
$T(n/5)$	{ 2. Recursively SELECT the median x of the $\lfloor n/5 \rfloor$ group medians to be the pivot.
$\Theta(n)$	3. Partition around the pivot x . Let $k = \text{rank}(x)$.
$T(3n/4)$	{ 4. if $i = k$ then return x elseif $i < k$ then recursively SELECT the i th smallest element in the lower part else recursively SELECT the $(i-k)$ th smallest element in the upper part

应用11:选择问题-算法改进

Solving the recurrence

$$T(n) = T\left(\frac{1}{5}n\right) + T\left(\frac{3}{4}n\right) + \Theta(n)$$

Substitution:

$$T(n) \leq cn$$

$$T(n) \leq \frac{1}{5}cn + \frac{3}{4}cn + \Theta(n)$$

$$= \frac{19}{20}cn + \Theta(n)$$

$$= cn - \left(\frac{1}{20}cn - \Theta(n)\right)$$

$$\leq cn ,$$

if c is chosen large enough to handle both the $\Theta(n)$ and the initial conditions.

应用11:选择问题-举例

- 考察如下情形: $r=5, n=27$, 并且 $a=[2, 6, 8, 1, 4, 10, 20, 6, 22, 11, 9, 8, 4, 3, 7, 8, 16, 11, 10, 8, 2, 14, 15, 1, 12, 5, 4]$ 。
 - 说明: 这27个元素可以被分为6组, 每组的中间元素分别为4, 11, 7, 10, 12和4, 这些中间元素的中间元素为7, 将其取为支点元素。
-

应用11:选择问题-重要结论

- 当按“中间的中间”规则选取支点元素时，以下结论为真：

若 $r=5$ ，且 a 中所有元素都不同，那么当 $n \geq 24$ 时，有 $\max\{|left|, |right|\} \leq 3n/4$ 。

有 $\left\lceil \frac{3}{2} \left\lfloor \frac{n}{5} \right\rfloor \right\rceil$ 多个中间元素不大于支点元素；

大于支点元素的数目有上界

$$n - \frac{3}{2} \left\lfloor \frac{n}{5} \right\rfloor \leq n - 1.5(n/5 - 4/5) = 0.7n + 1.2$$

- 因此，当 $n \geq 24$ 时， $0.7n + 1.2 \leq 3n/4$
 - $T(n) \leq T(n/5) + T(3n/4) + cn$. 用归纳法可证明当 $n \geq 24$ 时有 $T(n) \leq 20cn$ 。
-

应用11:选择问题-重要结论

- 分析 $r=7$ 时选择算法的时间复杂度。
- 当 $r=7$ 时，大于支点元素的个数不超过

$$\begin{aligned}n - 4 \left\lceil \frac{1}{2} \left\lfloor \frac{n}{7} \right\rfloor \right\rceil &\leq n - 2 \left\lfloor \frac{n}{7} \right\rfloor \\&\leq n - 2 \left(\frac{n}{7} - \frac{6}{7} \right) = \frac{5n}{7} + \frac{12}{7} \\&\leq \frac{5n}{6} \quad n \geq 15\end{aligned}$$

- 所以, $T(n) \leq T(n/7) + T(5n/6) + cn$, 当 $n > 14$ 时成立。
-

应用11:选择问题-重要结论

- 当 $r=9$, 当 $n \geq 90$ 时, 有 $\max\{|left|, |right|\} \leq 7n/8$
- 用于寻找第 k 个元素的时间 $T(n)$ 可按如下递归公式来计算:

$$T(n) = \begin{cases} c & n < 90 \\ T(\lceil n/9 \rceil) + T(\lfloor 7n/8 \rfloor) + cn & n \geq 90 \end{cases}$$

- 在上述递归公式中, 假设当 $n < 90$ 时使用任意的求解算法, 当 $n \geq 90$ 时, 采用“中间的中间”规则用分治法求解。利用归纳法可以证明, 当 $n \geq 90$ 时有 $T(n) \leq 72cn$ 。
-

应用11:选择问题-重要结论

- 分析当 r 取3时是否能在 $O(n)$ 时间内求解选择问题?
 - 当 $r=3$ 时, 找不到满足 $\alpha + 1/3 < 1$ 的正数 α 使得
$$n - 2 \left\lceil \frac{1}{2} \left\lfloor \frac{n}{3} \right\rfloor \right\rceil \leq \alpha n .$$
 - 在 $r=7$ 时可找到 $\alpha=5/6$, $r=5$ 时 $\alpha=3/4$, $r=9$ 时可找到 $\alpha=7/8$.
 - 所以, 当 $r=3$ 时, 算法不能保证在 $O(n)$ 时间内求解.
-

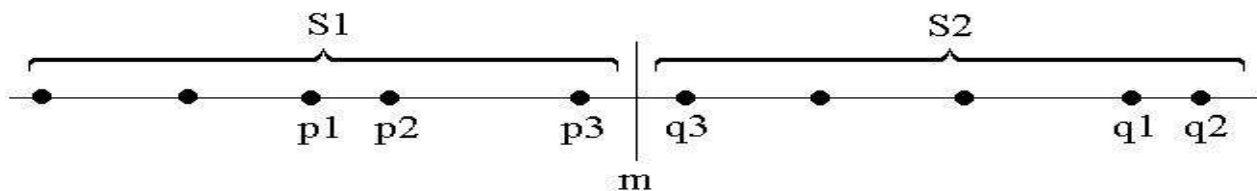
应用12：最近点对问题

- 问题描述：给定平面上 n 个点，找其中的一对点，使得在 n 个点所组成的所有点对中，该点对间的距离最小。
 - 说明：
 - 严格来讲，最接近点对可能多于一对，为简便起见，我们只找其中的一对作为问题的解。
 - 一个简单的做法是将每一个点与其他 $n-1$ 个点的距离算出，找出最小距离的点对即可。该方法的时间复杂度是 $T(n)=n(n-1)/2+n=O(n^2)$ ，效率较低。
-

应用12：最近点对问题-一维情形

◆考虑一维的情形。此时， S 中的 n 个点退化为 x 轴上的 n 个实数 $x_1, x_2, \dots, x_n$ 。最接近点对即为这 n 个实数中相差最小的2个实数。

- 假定平面上 n 个点的集合 S ，找其中的一对点，使得在 n 个点组成的所有点对中，该点对间的距离最小。
- 设我们用 x 轴上某个点 m 将 S 划分为2个子集 S_1 和 S_2 ，基于平衡子问题的思想，用 S 中各点坐标的中位数来作分割点。
- 递归地在 S_1 和 S_2 上找出其最接近点对 $\{p_1, p_2\}$ 和 $\{q_1, q_2\}$ ，并设 $d = \min\{|p_1 - p_2|, |q_1 - q_2|\}$ ， S 中的最接近点对或者是 $\{p_1, p_2\}$ ，或者是 $\{q_1, q_2\}$ ，或者是某个 $\{p_3, q_3\}$ ，其中 $p_3 \in S_1$ 且 $q_3 \in S_2$ 。
- 能否在线性时间内找到 p_3, q_3 ？



应用12：最近点对问题-一维情形

□ 能否在线性时间内找到 p_3, q_3 ?

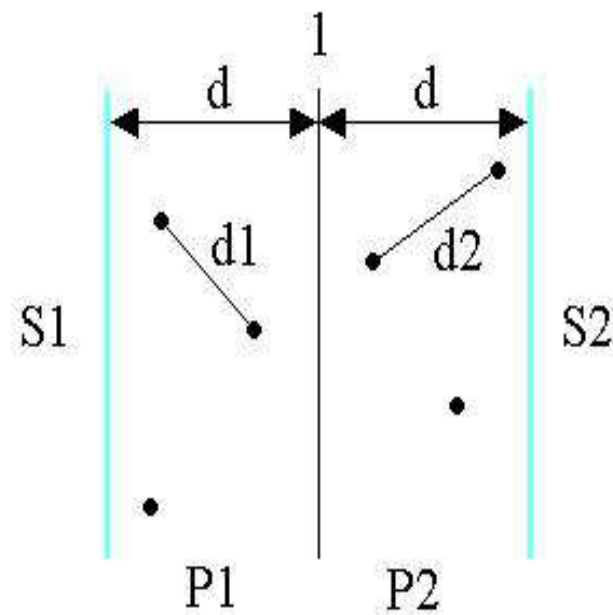
- 如果 S 的最接近点对是 $\{p_3, q_3\}$ ，即 $|p_3 - q_3| < d$ ，则 p_3 和 q_3 两者与 m 的距离不超过 d ，即 $p_3 \in (m-d, m]$ ， $q_3 \in (m, m+d]$ 。
- 由于在 S_1 中，每个长度为 d 的半闭区间至多包含一个点（否则必有两点距离小于 d ），并且 m 是 S_1 和 S_2 的分割点，因此 $(m-d, m]$ 中至多包含 S 中的一个点。由图可以看出，如果 $(m-d, m]$ 中有 S 中的点，则此点就是 S_1 中最大点。
- 因此，我们用线性时间就能找到区间 $(m-d, m]$ 和 $(m, m+d]$ 中所有点，即 p_3 和 q_3 。从而我们用线性时间就可以将 S_1 的解和 S_2 的解合并成为 S 的解。

□ 注意：分割点 m 的选取不当，会造成 $|S_i|=1$ ， $|S_j|=n-1$ 的情形，使得 $T(n) = T(n-1) + O(n) = O(n^2)$ 。这种情形可以通过“平衡子问题”方法加以解决：选取各点坐标的中位数作分割点。

应用12：最近点对问题-二维情形

□ 下面来考虑二维的情形

- 选取一垂直线 $l: x=m$ 来作为分割直线，其中 m 为 S 中各点 x 坐标的中位数，由此将 S 分割为 S_1 和 S_2 。
- 递归地在 S_1 和 S_2 上找出其最小距离 d_1 和 d_2 ，并设 $d = \min\{d_1, d_2\}$ ， S 中的最接近点对或者是 d ，或者是某个 $\{p, q\}$ ，其中 $p \in S_1$ 且 $q \in S_2$ 。
- 能否在线性时间内找到 p, q ？

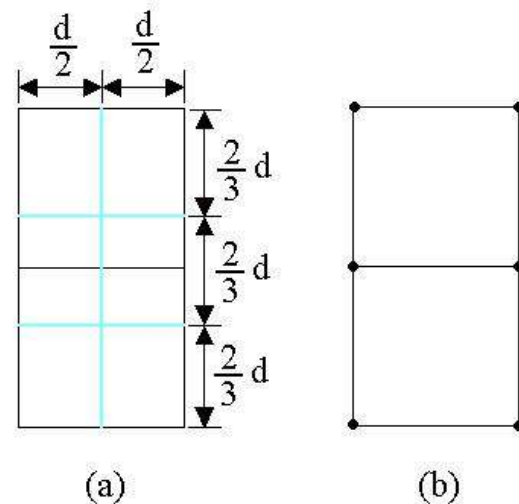
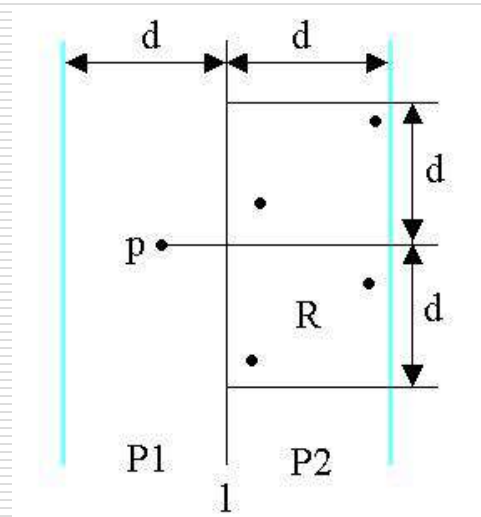


应用12：最近点对问题-二维情形

□ 能否在线性时间内找到 p_3, q_3 ?

- ◆ 考虑P1中任意一点 p ，它若与P2中的点 q 构成最接近点对的候选者，则必有 $\text{distance}(p, q) < d$ 。满足这个条件的P2中的点一定落在一个 $d \times 2d$ 的矩形R中。
- ◆ 由 d 的意义可知，P2中任何2个S中的点的距离都不小于 d 。由此可以推出矩形R中最多只有6个S中的点。
- ◆ 因此，在分治法的合并步骤中最多只需要检查 $6 \times n/2 = 3n$ 个候选者

□ 注意：符号P1和P2不同于S1和S2



应用12：最近点对问题-二维情形

□ 附：矩形R中最多只有6个S中的点的证明

□ 证明：将矩形R的长为 $2d$ 的边3等分，

将它的长为 d 的边2等分，由此导出6个 $(d/2) \times (2d/3)$ 的矩形。若矩形R中有多于6个S中的点，易知至少有一个 $(d/2) \times (2d/3)$ 的小矩形中有2个以上S中的点。设 u, v 是位于同一小矩形中的2个点，则

$$(x(u)-x(v))^2 + (y(u)-y(v))^2 \leq (d/2)^2 + (2d/3)^2 = 25d^2/36$$

$\text{distance}(u, v) < d$ 。这与 d 是矩形R中最小距离相矛盾。

应用12：最近点对问题-二维情形

- 为了确切地知道要检查哪6个点，可以将 p 和 P_2 中所有 S_2 的点投影到垂直线 l 上。
 - 由于能与 p 点一起构成最接近点对候选者的 S_2 中点一定在矩形 R 中，所以它们在直线 l 上的投影点距 p 在 l 上投影点的距离小于 d 。由上面的分析可知，这种投影点最多只有6个。
 - 因此，若将 P_1 和 P_2 中所有 S 中点按其 y 坐标排好序，则对 P_1 中所有点，对排好序的点列作一次扫描，就可以找出所有最接近点对的候选者。
 - 对 P_1 中每一点最多只要检查 P_2 中排好序的相继6个点。
-

应用12：最近点对问题-算法

```
public static double cpair2(S)
```

```
{
```

```
    n=|S|;
```

```
    if (n < 2) return    ;
```

```
1. m=S中各点x间坐标的中位数;
```

```
    构造S1和S2;
```

```
    //S1={p∈S|x(p)≤m},
```

```
    S2={p∈S|x(p)>m}
```

```
2. d1=cpair2(S1);
```

```
    d2=cpair2(S2);
```

```
3. dm=min(d1,d2);
```

4. 设P1是S1中距垂直分割线l的距离在dm之内的所有点组成的集合;

P2是S2中距分割线l的距离在dm之内所有点组成的集合;

将P1和P2中点依其y坐标值排序;

并设X和Y是相应的已排好序的点列;

5. 通过扫描X以及对于X中每个点检查Y中与其距离在dm之内的所有点(最多6个)可以完成合并;

当X中的扫描指针逐次向上移动时，Y中的扫描指针可在宽为2dm的区间内移动; 设dl是按这种扫描方式找到的点对间的最小距离;

```
6. d=min(dm,dl);  
    return d;
```

```
}
```

应用12：最近点对问题-算法复杂度

□ 时间复杂度函数分析函数：

$$\square T(n) = \begin{cases} O(1) & n < 4 \\ 2T(n/2) + O(n) & n \geq 4 \end{cases}$$

□ 使用Master方法，算法复杂度分析结果：
 $T(n)=O(n\log n)$

应用13：循环赛日程安排

- 设有 $n=2^k$ 个运动员要进行网球循环赛，现在要设计一个满足以下要求的比赛日程表：
 - (1) 每个选手必须与其他 $n-1$ 个选手各赛一次；
 - (2) 每个选手一天只能赛一次；
 - (3) 循环赛一共进行 $n-1$ 天。
-

应用13：循环赛日程安排

- 按分治策略，可以将所有的选手分为两半， n 个选手的比赛日程表可以通过为 $n/2$ 个选手设计的比赛日程表来决定。
 - 递归地使用这种一分为二的策略对选手进行划分，直到只剩下2个选手时，比赛日程表的制定就变得很简单了。
 - 这时只要让这两个选手进行比赛就可以了。
-

应用13：循环赛日程安排-举例

- 右表是8个选手的比赛日程表。
- 左上角与左下角的2小块分别为选手1至选手4以及选手5至选手8前3天的比赛日程。
- 将左上角小块的数字复制到右下角，左下角的数字复制到右上角，就排好了选手们后4天的比赛日程。

1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1

应用13：循环赛日程安排-算法

```
❑ void gametable(int k)
❑ {
❑     int a[100][100];
❑     int n, temp, i, j, p, t;
❑     n=2;//两个参赛选手日程
❑     a[1][1]=1;a[1][2]=2;
❑     a[2][1]=2;a[2][2]=1;
❑     for(t=1; t<k; t++)//迭代处理
        , 依次处理 $2^n \dots 2^k$ 个选手的
        比赛日程
❑     {
❑         temp=n; n=n*2;//填左下角
        元素
❑         for(i=temp+1; i<=n; i++)
```

```
❑         for(j=1; j<=temp; j++)
❑             a[i][j]=a[i-temp][j]+temp;
❑         //左下角和左上角元素的对应关
        系
❑         for(i=1; i<=temp; i++)
❑             //将左下角元素抄到右上角
            for(j=temp+1; j<=n; j++)
❑                 a[i][j]=a[i+temp][(j+temp
        )%n];
❑         for(i=temp+1; i<=n; i++)
❑             //将左上角元素抄到右下角
            for(j=temp+1; j<=n; j++)
❑                 a[i][j]=a[i-temp][j-
        temp];
❑     }
```

内容回顾

□ 分治算法的基本思想

□ 分治算法的应用

- 1) 找伪币问题 2) 计算 a^n 3) 求Fibonacci数
- 4) 金块问题 5) 大整数乘法 6) 矩阵乘法
- 7) VLSI 布线 8) 残缺棋盘覆盖 9) 归并排序
- 10) 快速排序 11) 选择问题 12) 最接近点对问题
- 13) 循环赛日程

□ 复杂性的上限、下限

- 最大最小问题的下限
 - 排序算法的下限
-

复杂性的上限、下限

□ 复杂性的上限

- 当且仅当某个问题至少有一个复杂性为 $O(f(n))$ 的求解算法时，存在一个复杂性的上限(upper bound) $f(n)$
 - 证明一个问题复杂性上限为 $f(n)$ 的一种方法是设计一个复杂性为 $O(f(n))$ 的算法
 - 例，在发现Strassen矩阵乘法之前，矩阵乘法的复杂性上限为 n^3 ，Strassen算法的发现使复杂性上限降为 $n^{2.373}$
-

复杂性的上限、下限

□ 复杂性的下限

- 当且仅当求解一个问题**所有算法**的复杂性均为 $\Omega(f(n))$ 时，则称 $f(n)$ 为该问题的**复杂性的下限**（lower bound）
 - 为了确定一个问题的复杂性下限 $g(n)$ ，必须**证明该问题的每一个求解算法的复杂性均为 $\Omega(g(n))$**
 - 要得到这样一个结论相当困难，因为不可能考察所有的求解算法
 - 对于大多数问题，可以建立一个基于输入和/或输出数目的简单下限。
 - 例如：对 n 个元素进行排序的算法的复杂性为 $\Omega(n)$ ，因为所有的算法对每一个元素都必须检查至少一遍，否则未检查的元素可能会排列在错误的位置上
 - 例如：每一个计算两个 $n \times n$ 矩阵乘法的算法都有复杂性 $\Omega(n^2)$ 。因为结果矩阵中有 n^2 个元素并且产生每个元素所需要的时间为 $\Omega(1)$ 。
-

比较算法

- 所谓比较算法是指算法的操作主要限于元素比较和元素移动。
 - 本节将建立本章所介绍的两个分治法求解的问题的确切下限——寻找 n 个元素中的最小最大值问题以及排序问题。
 - 对于这两个问题，仅考察基于关键字比较的算法。
-

最大最小问题算法复杂度下限

- 在本章求解金块问题的应用中，按分治法设计的在 n 个元素中找最大值与最小值的算法，该算法需做 $\lceil 3n/2 \rceil - 2$ 次元素比较。
 - 以下证明：解决该问题的任何一个基于比较的算法，至少需要做 $\lceil 3n/2 \rceil - 2$ 次。
 - 问题下界指解决该问题的所有算法的最坏情形复杂度的最小值(min-max):
 - $\max$ 对所有实例; $\min$ 对所有算法.
-

最大最小问题算法复杂度下限

□ 状态空间法 (State-space method)

- 状态空间方法要求首先定义算法的三种状态：起始状态、中间状态和完成状态，并需描述每次比较状态如何转换，然后确定从起始状态到完成状态worst case至少需要的状态转换数。
 - 构造一个产生最坏情形的输入, 我们称之为敌手(adversary)。
 - 假设 n 个元素互不相同。
-

max-min问题的状态空间

- 算法状态可用元组(a, b, c, d)来描述,
 - a 表示算法需考察的候选的最大和最小元素的个数(尚未参加过比较的那些元素)
 - b 表示不再做为最小候选但仍作为最大候选的元素个数(只胜未败)
 - c 是不再做为最大候选但仍做为最小候选的元素个数(只败未胜)
 - d 是被确定为既非最大也非最小的元素个数。
 - A, B, C, D 代表上述各种元素的集合。
-

证明（过程）

- 算法启动时：起始状态为 $(n, 0, 0, 0)$
 - 算法结束时：完成状态为 $(0, 1, 1, n-2)$
 - 在比较元素的过程中算法状态发生变化
 - 当 A 中的两个元素比较时，较小的元素放入 C ，较大的元素放入 B (根据假设所有元素都不相同，因此不会出现相等的情形),引起下面状态转换： $(a, b, c, d) \rightarrow (a-2, b+1, c+1, d)$
-

证明（过程）

其他可能的状态转换如下：

□ B 中元素比较之后，状态转换为：

$$(a, b, c, d) \rightarrow (a, b-1, c, d+1)$$

□ C 中元素比较之后，状态转换为：

$$(a, b, c, d) \rightarrow (a, b, c-1, d+1)$$

□ B 与 C 中元素比较：

$$(a, b, c, d) \rightarrow (a, b-1, c-1, d+2)$$

$$*(a, b, c, d) \rightarrow (a, b, c, d)$$

□ $*$ 代表最坏情形状态变换

证明（过程）

- A 中元素与 B 中元素进行比较，状态转换为：
 $(a, b, c, d) \rightarrow (a-1, b, c, d+1)$ (A 中元素 $>$ B 中元素)
* $(a, b, c, d) \rightarrow (a-1, b, c+1, d)$ (A 中元素 $<$ B 中元素)
 - A 中元素与 C 中元素进行比较，状态转换为：
 $(a, b, c, d) \rightarrow (a-1, b, c, d+1)$ (A 中元素 $<$ C 中元素)
* $(a, b, c, d) \rightarrow (a-1, b+1, c, d)$ (A 中元素 $>$ C 中元素)
-

证明（过程）

□ A与D中元素比较状态转换：

$(a,b,c,d) \rightarrow (a-1,b+1,c,d)$

$(a,b,c,d) \rightarrow (a-1,b,c+1,d)$

□ 因我们分析最坏情形复杂度的下界，所以A与B或C中元素比较及B与C中元素比较时应考虑最坏情形：

$*(a,b,c,d) \rightarrow (a-1,b,c+1,d)$

$*(a,b,c,d) \rightarrow (a-1,b+1,c,d)$

$*(a,b,c,d) \rightarrow (a,b,c,d)$

□ 构造最坏情形输入使得算法在此输入下d的值增加最慢：

■ A与B的元素比较令B的元素胜：

如实际上 $x > y$ ，因y从未输过，重新取y的值使 $y > x$ 。在新的输入下，以前比赛过程不会改变。

■ A的元素与C的元素比较令C的元素败；

■ B与C的元素比较令B的元素胜。

证明（过程）

- 其他比较无实质意义，例如D中元素x与B中元素y比较且 $x > y$ ，则改变y的值使其大于x，不会影响以前的比赛，d值不变。D与C中元素比较同样处理。
 - 上面构造的输入使得从初始状态到结束状态至少要做的状态转数：
 - d从0增至 $n-2$ ，至少要 $n-2$ 次比较，而且这些比较不会引起a减少（只能B中或C中元素自己比）；
 - a从n减至0，至少要做：当 $n=2m$ （为偶数）时为m次；当 $n=2m+1$ 时为 $m+1$ 次；
 - 所以至少要做 $\lceil 3n/2 \rceil - 2$ 次比较
-

状态空间法证明结果

- 任何求最大最小问题的算法从起始状态到完成状态所用比较次数不可少于 $\lceil 3n/2 \rceil - 2$
 - $\lceil 3n/2 \rceil - 2$ 是所有基于比较的求最大最小算法所需比较次数的下界。
 - 在应用4的金块问题中，分治算法是解决最大最小问题的最优算法
-

排序算法的下限

How fast can we sort?

All the sorting algorithms we have seen so far are **comparison sorts**: only use comparisons to determine the relative order of elements.

- *E.g.*, insertion sort, merge sort, quicksort, heapsort.

The best worst-case running time that we've seen for comparison sorting is $O(n \lg n)$.

Is $O(n \lg n)$ the best we can do?

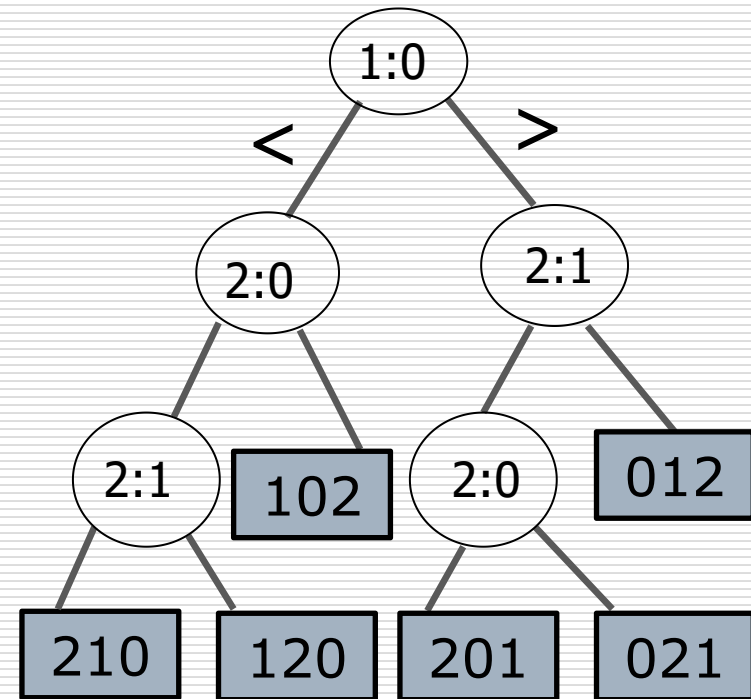
Decision trees can help us answer this question.

排序算法的下限

- ❑ 可用决策树（decision-tree）来证明下限。
 - ❑ 证明过程中用树来模拟算法的执行过程。
 - ❑ 对于树的每个内部节点，算法执行一次比较并根据比较结果移向它的某一子节点。
 - ❑ 算法在叶节点处终止。
-

InsertionSort 的决策树 ($n=3$)

- 右图给出了对三个元素 $a[0:2]$ 使用InsertionSort排序时的决策树。
- 每个内部节点有一个 $i:j$ 的标志, 表示 $a[i]$ 与 $a[j]$ 进行比较。
 - 如果 $a[i] < a[j]$, 算法移向左分枝;
 - 如果 $a[i] > a[j]$, 移向右分枝。
 - 因为元素互不相同, 所以 $a[i] = a[j]$ 不会发生。
- 叶节点标出了所产生的排序。
 - 图中最左路径代表: $a[1] < a[0]$, $a[2] < a[0]$, $a[2] < a[1]$, 因此最左叶节点为 $(a[2], a[1], a[0])$ 。



决策树说明

- 假定输入是 $1, \dots, n$ 的一个排列,决策树中每个叶节点代表对输入排列的排序结果.
 - 例如,对输入(102)排序的结果为($a[1]a[0]a[2]$).
 - 正确的排序算法对于 n 个输入排列必须产生 $n!$ 排序结果,因此决策树有 $n!$ 个叶节点.
 - 有 n 个叶节点的二叉树的深度至少为 $\log n$
 - 因此决策树的高度至少为 $\log n! = \Theta(n \log n)$
 - 每一个比较排序算法在最坏情况下至少要进行 $\Omega(n \log n)$ 次比较.
 - 每个有 $n!$ 个叶节点的决策树的平均外路长度为 $\Omega(n \log n)$;
 - 所以每个比较排序算法的平均复杂性也是 $\Omega(n \log n)$
-

结论

- 由前面的证明可以看出，堆排序、归并排序在最坏情况下有较好的性能（针对渐进复杂性而言）
 - 堆排序、归并排序、快速排序在平均情况下性能较优。
-

补充思考题：逆序对问题

- 给定自然数 $1, \dots, n$ 的一个排列，例如， $(1, 3, 4, 2, 5)$ ，如果 $j > i$ 但 j 排在 i 的前面则称 (j, i) 为该排列的一个逆序。在上例中 $(3, 2)$ ， $(4, 2)$ 为该排列的逆序。该排列总共有 **2** 个逆序对。
- 试用分治法设计一个计算给定排列的逆序对总数的算法，并分析算法的复杂度

提示：可对归并排序算法进行修改得到。
